

西南石油大学

本科毕业设计（论文）



题 目	<u>图书馆自习座位智能预约系统</u>		
学生姓名	<u>陈信豪</u>	学 号	<u>202231061133</u>
教学院系	<u>计算机与软件学院</u>		
专业年级	<u>物联网工程 2022 级</u>		
指导教师	<u>梁宗文</u>	职 称	<u>讲师</u>
单 位	<u>西南石油大学</u>		
辅导教师	职 称		
单 位			

完成日期 2026 年 05 月 11 日

Southwest Petroleum University

Graduation Thesis



Intelligent Reservation System for Library Study Seats

Grade: 2022

Name: Chen Xinhao

Specialty: Internet of Things Engineering

Instructor: Liang Zongwen

School of Computer Science and Software Engineering

2026-5

摘 要

随着高等教育规模的持续扩大，高校图书馆座位资源的供需矛盾日益凸显。根据教育部统计数据，全国各类高等教育在学总规模已达 4846 万人，多数高校生均座位数低于 0.25 座的国家标准。在考试周、考研冲刺期等高峰时段，“一座难求”的现象屡见不鲜。传统的人工管理方式存在信息滞后、占座现象严重、座位利用率低等问题；现有智能方案多采用有线部署或高成本传感器，施工难度大、经济可行性低。因此，研究一种低成本、易部署、功能完善的图书馆自习座位智能管理系统具有重要的现实意义。

针对上述问题，本文设计并实现了一套图书馆自习座位智能预约系统。系统采用“端-云-端”三层架构。硬件层选用 ESP32-38Pin-CP2102 微控制器配合 HC-SR501 红外传感器进行座位状态检测，通过 MQTT 协议完成数据传输，单座位器件成本控制在约 35 元。后端服务层基于 NestJS 框架构建，采用 TypeORM 操作 MySQL 数据库，Redis 实现分布式缓存与原子锁；通过 MQTT Broker 接收硬件数据上报，利用 Socket.io 房间机制实现前端实时推送。前端应用层使用 Taro 跨端框架和 React 实现微信小程序端。系统核心功能包括：座位状态实时监测、智能座位推荐（基于空闲时长、历史使用率与用户偏好的多维度评分算法，结合五级渐进式条件放宽策略）、犹豫期锁定机制（IN_JUDGE 状态限时确认防止恶意占座）、多模态签到（HMAC-SHA256 签名二维码、GPS 位置验证、传感器自动检测）、设备指纹防账号共享、信誉分管理与暂离超时自动释放。

经功能测试、性能测试和边界安全测试验证，系统各模块运行稳定，API 平均响应时间 45 120ms，端到端延迟控制在 1 秒以内，满足设计需求。

关键词：图书馆自习座位管理；ESP32；MQTT；智能推荐；NestJS；微信小程序

Abstract

With the continuous expansion of higher education, the supply-demand contradiction of library seat resources in universities has become increasingly prominent. According to official statistics, the total enrollment in higher education has reached 48.46 million, and the average number of seats per student in most universities falls below the national standard of 0.25. During peak periods such as final exam weeks, the scarcity of library seats is a widespread issue. Traditional manual management methods suffer from problems such as lagging information updates, prevalent seat-hogging phenomena, and low seat utilization rates. Existing intelligent solutions mostly rely on wired deployment or expensive sensors, resulting in high construction costs and low economic feasibility. Therefore, research on a low-cost, easy-to-deploy, and fully functional intelligent library seat management system is of great practical significance.

To address the above issues, this thesis designs and implements an intelligent reservation system for library study seats with a three-tier architecture. The hardware layer uses ESP32-38Pin-CP2102 microcontroller with HC-SR501 infrared sensor to detect seat status and transmits data through MQTT protocol, with a per-seat device cost of approximately ¥35. The backend service layer is based on NestJS framework, uses TypeORM to operate MySQL database, and employs Redis for distributed caching and atomic locking. It receives hardware data via MQTT Broker and implements real-time push to frontend through Socket.io room mechanism. The frontend application layer uses Taro cross-platform framework and React to implement WeChat mini-program. Core functions include real-time seat status monitoring, intelligent seat recommendation based on a multi-dimensional scoring algorithm combining idle time, historical usage rate, and user preference with a five-level gradual condition relaxation strategy, hesitation period locking mechanism to prevent malicious seat occupation, multi-modal check-in including HMAC-SHA256 signed QR code, GPS location

verification, and sensor automatic detection, device fingerprinting for anti-account-sharing, credit score management, and automatic release after temporary leave timeout.

Verified through functional testing, performance testing, and boundary security testing, all modules of the system operate stably. The API average response time ranges from 45ms to 120ms, and the end-to-end delay is controlled within one second, meeting the design requirements.

Keywords: Library Seat Management; ESP32; MQTT; Intelligent Recommendation; NestJS; WeChat Mini Program

目 录

1 绪论	13
1.1 研究背景与意义	13
1.1.1 研究背景	13
1.1.2 研究意义	14
1.2 国内外研究现状	14
1.2.1 智能座位管理系统研究现状	14
1.3 低功耗无线通信技术选型分析	15
1.3.1 低功耗无线通信技术对比	15
1.3.2 技术选型决策	16
1.4 本文研究的主要内容	16
1.4.1 硬件层设计与实现	16
1.4.2 智能推荐与预约机制设计	16
1.4.3 多模态签到机制设计	17
1.4.4 防作弊与信誉体系设计	17
1.4.5 实时通信机制设计	17
2 相关技术理论	19
2.1 物联网通信技术	19
2.1.1 MQTT 协议	19
2.1.1.1 为什么选 MQTT	19
2.1.1.2 实际应用中的问题	19
2.1.2 WebSocket 与 Socket.io	19
2.1.2.1 选择 Socket.io 而非原生 WebSocket 的原因	20
2.2 后端技术栈	20
2.2.1 NestJS 框架	20
2.2.1.1 NestJS 与 Express 的技术对比	20

2.2.1.2	为什么选择 NestJS	21
2.2.2	MySQL 数据库	21
2.2.2.1	选择 MySQL 的理由	21
2.2.3	TypeORM	21
2.2.3.1	为什么选 TypeORM	21
2.2.4	Redis	21
2.2.4.1	Redis 的应用场景	21
2.3	前端技术栈	22
2.3.1	React 18	22
2.3.2	Taro 跨端框架	22
2.3.3	轻量状态管理	22
2.4	硬件技术	22
2.4.1	红外人体传感技术	22
2.4.2	ESP32 微控制器	22
2.5	本章小结	23
3	系统设计	24
3.1	系统需求分析	24
3.1.1	系统总体目标	24
3.1.1.1	实时性目标	24
3.1.1.2	易用性目标	24
3.1.1.3	成本目标	24
3.1.2	功能需求分析	24
3.1.2.1	座位状态实时监测	25
3.1.2.2	智能座位推荐	25
3.1.2.3	座位预约与签到	25
3.1.2.4	信誉分管理	26
3.1.2.5	用户认证与权限管理	26

3.1.3	非功能需求分析	26
3.1.3.1	性能需求	26
3.1.3.2	可靠性需求	26
3.1.3.3	安全性需求	26
3.1.3.4	可扩展性需求	26
3.2	系统总体架构设计	26
3.2.1	架构的演进过程	26
3.2.1.1	为什么是三层	27
3.2.2	技术选型的最终决定	27
3.2.3	Monorepo 项目架构	27
3.3	硬件层设计	28
3.3.1	ESP32 单芯片方案的设计思路	28
3.3.1.1	为什么选用 ESP32	28
3.3.1.2	硬件选型明细	28
3.3.1.3	工作流程	28
3.3.1.4	边缘侧信号处理	29
3.3.2	数据传输效率分析	29
3.4	后端架构设计	29
3.4.1	分层架构	30
3.4.1.1	各层职责	30
3.4.2	数据流向	30
3.5	数据库设计	30
3.5.1	E-R 图	30
3.5.2	逻辑设计	31
3.6	API 接口设计	32
3.6.1	RESTful API 规范	32
3.6.2	后端中间件体系	32

3.6.3 核心接口定义	32
3.7 核心算法设计	32
3.7.1 智能座位评分算法	32
3.7.1.1 算法描述	32
3.7.1.2 五级渐进式条件放宽策略	33
3.7.2 预约冲突检测算法	33
3.7.2.1 算法描述	33
3.7.2.2 状态互斥判断	33
3.7.3 时间窗口观察算法（传感器自动签到）	33
3.7.3.1 算法描述	33
3.7.3.2 伪代码	34
3.7.4 信誉分计算算法	34
3.7.4.1 算法描述	34
3.8 本章小结	34
4 系统实现	36
4.1 开发环境与项目结构	36
4.1.1 开发工具与环境配置	36
4.1.2 Monorepo 项目结构	36
4.2 硬件层实现	36
4.2.1 ESP32 红外检测代码	36
4.2.2 硬件连接与调试	36
4.2.3 OLED 状态显示屏驱动	37
4.3 后端服务层实现	38
4.3.1 MQTT Broker 搭建	38
4.3.2 MQTT 显示数据推送	38
4.3.3 Socket.io 实时推送	39
4.3.4 Redis + Lua 分布式锁	40

4.3.5	用户认证	41
4.3.6	座位预约系统	41
4.3.6.1	预约状态流转	41
4.3.6.2	犹豫期锁定机制	42
4.3.7	HMAC-SHA256 签名二维码	42
4.3.8	位置验证签到	43
4.3.9	设备指纹防账号共享	43
4.3.10	信誉分管理	44
4.3.11	暂离超时处理	44
4.4	前端应用层实现	44
4.4.1	Taro 小程序端	44
4.4.2	智能预约页面	45
4.4.3	系统主要界面展示	46
4.4.4	实时消息处理时序	46
4.5	数据库物理设计	47
4.5.1	数据表结构	47
4.5.2	主外键对应关系	49
4.5.3	索引设计	50
4.6	本章小结	50
5	系统测试	51
5.1	测试环境与方法	51
5.1.1	测试环境配置	51
5.1.2	测试方法	51
5.2	功能测试	51
5.2.1	系统操作流程测试	51
5.2.2	用户认证测试	56
5.2.3	座位状态监测测试	56

5.2.4	智能推荐测试	56
5.2.5	预约功能测试	57
5.2.6	签到功能测试	57
5.2.7	信誉分管理测试	57
5.2.8	暂离超时释放测试	57
5.3	性能测试	58
5.3.1	响应时间测试	58
5.3.2	实时性测试	59
5.3.3	并发测试	60
5.4	边界安全测试	61
5.5	测试结果分析	62
5.5.1	测试中发现的问题	62
5.5.2	测试范围说明	63
5.6	本章小结	63
6	总结与展望	64
6.1	工作总结	64
6.1.1	完成的工作	64
6.1.2	创新点	64
6.2	存在的不足	65
6.3	未来展望	65
6.4	本章小结	66
	参考文献	67

绪论

1.1 研究背景与意义

1.1.1 研究背景

高校图书馆座位资源紧张的现状

近年来,我国高等教育规模持续扩大。根据教育部《2024 年全国教育事业发展统计公报》^[1],全国各类高等教育在学总规模达 4846 万人,多数高校生均座位数低于教育部规定的 0.25 座标准^[2]。以西南石油大学为例,在校学生约 4.2 万人,图书馆阅览座位约 5,200 个,生均约 0.12 座。在考试周、考研冲刺期等高峰时段,座位供需矛盾尤为突出,“一座难求”的现象屡见不鲜。

传统管理方式的局限性

通过文献调研和实地观察,高校图书馆座位管理主要存在以下问题:

(1) 信息不对称问题。传统的座位管理方式缺乏实时信息发布渠道,学生无法提前了解座位占用情况,只能“盲目前往”。这种信息不对称导致学生花费大量时间寻找座位,降低了学习效率。

(2) 占座现象普遍。由于座位资源紧张且信息不透明,部分学生倾向于提前占位以“确保”学习位置。相关研究显示,高校图书馆占座现象普遍存在,部分座位被书本等物品占据但长时间无人使用,造成资源浪费。

(3) 签到核验困难。传统预约系统缺乏有效的签到机制,用户预约后不到场或代他人签到的情况难以监管,导致预约资源被浪费。

(4) 现有智能方案成本较高。目前市场上已有的智能座位管理系统,多采用压力传感器配合有线网络或 ZigBee 等无线通信技术。有线方案需要破墙布线,施工成本高;无线方案模块成本普遍在 50-100 元不等,大规模改造成本较高。

物联网技术带来的新机遇

近年来,物联网技术的快速发展^[3]为解决上述问题提供了新的可能。低功耗无线通信技术日趋成熟,硬件成本大幅下降;开源实时通信框架降低了开发门槛;跨端开发框架使得一套代码部署多端成为现实。这些技术进步为构建低成本、易部署的智能座位预约系统创造了条件,也构成了本研究的出发点。

1.1.2 研究意义

理论意义

本研究涉及 IoT 感知技术、实时通信架构设计、智能推荐算法、分布式并发控制等技术问题，对相关领域的技术应用具有一定参考价值。

(1) 智能推荐算法设计。本研究设计了基于空闲时长、历史使用率与用户偏好的多维度座位评分算法，结合五级渐进式条件放宽策略，实现了兼顾用户偏好与系统负载均衡的智能座位分配。

(2) 分布式并发控制。本研究采用 Redis+Lua 原子脚本实现座位预约的分布式锁，解决了高并发场景下同座位的重复预约问题，并支持同用户重入，为类似预约系统的并发控制提供了参考方案。

(3) 多模态签到机制。本研究设计了扫码签到、位置验证签到和传感器自动检测三种签到方式，并通过 HMAC-SHA256 签名二维码防止伪造，为物联网场景的可靠身份核验提供了实践参考。

实践意义

本研究设计实现的系统如能落地应用，可产生以下实际价值：

(1) 对学生而言，实时座位信息查询功能可避免“白跑一趟”的情况；智能推荐功能可根据个人偏好快速分配最优座位；信誉分机制可规范预约行为，减少恶意占座。

(2) 对图书馆管理而言，实时监测功能可发现“占而不用”的情况，配合预约签到机制可提升座位资源利用率；数据统计功能可为资源配置决策提供依据。

(3) 部署优化，本研究采用的 ESP32 单芯片方案集成 WiFi 与 MCU，配合 SSD1306 OLED 状态显示屏，无需额外通信模块，单座位器件成本控制在约¥40 以内，无线部署无需布线，为大规模推广应用提供了可行路径。

1.2 国内外研究现状

1.2.1 智能座位管理系统研究现状

国外研究现状

欧美高校在智慧校园领域起步较早。Matuska 等^[4]设计了基于超声波距离传感器和 NodeMCU 的教室座位占用检测系统，支持 MQTT 传输并讨论了隐私优势；部分高

校探索了座椅内置压力传感器的检测方案^[5]，精度较高但部署维护成本高；日本东京大学采用 IC 卡门禁系统管理入馆权限，但未实现座位级实时监测。Budimir 等^[6]探索了基于物联网的运动检测系统用于智慧校园，验证了 IoT 传感器在室内环境监测中的可行性。国外方案普遍存在成本较高或适用性受限的问题，难以直接应用于国内高校场景。

国内研究现状

国内研究起步较晚但发展迅速。清华大学采用校园一卡通联动方案实现座位统计与预约，但依赖主动刷卡；浙江大学开发了基于微信公众号的座位预约系统，缺乏实时监测能力；电子科技大学探索了基于单片机和传感器的自动化检测方案，在通信稳定性和大规模部署方面仍面临挑战。现有研究主要集中在图书馆场景，方案在成本控制和功能完整性方面仍有提升空间。

现有系统的共性问题

综合国内外研究现状，现有方案普遍存在以下问题，如表表 1 所示。

问题	具体表现	影响分析
成本较高	单座位改造大多超过 100 元	大规模推广经济可行性低
实时性不足	部分系统更新延迟超过 1 分钟	用户体验较差
部署复杂	有线方案需破墙布线	施工周期长，学校改造意愿低
功能单一	仅具备座位查询功能	无法满足多样化需求
并发安全弱	高并发下存在重复预约漏洞	数据一致性与公平性受损

表 1 现有智能座位管理系统的共性问题

上述问题也是本研究重点解决的方面。

1.3 低功耗无线通信技术选型分析

1.3.1 低功耗无线通信技术对比

针对座位监测场景节点数量多、数据量小、功耗敏感的特点，对主流低功耗无线通信技术进行了对比分析。ZigBee 支持 mesh 组网但协议栈复杂、模块成本 30–50 元；BLE 手机普及率高但单网关连接数有限；LoRa 传输距离远但室内优势无法发挥、成本高；NB-IoT 基于运营商网络但需流量费、延迟高；ESP-NOW 无需 WiFi 握手、延迟毫秒级，但单网关限配约 20 个节点。综合对比后，ESP32 自带 WiFi 功能，可直接通过 MQTT

协议接入现有校园网络，无需额外网关设备，在成本和部署便利性方面最适合本系统场景。

1.3.2 技术选型决策

考虑时间约束和成本控制，本研究采用“原型验证+生产优化”的分步策略：当前实现阶段采用 ESP32 单芯片方案（单座位器件成本约¥40），该方案将微控制器与 WiFi 通信集成在同一芯片上，配合 OLED 状态显示屏实现现场交互，简化了硬件设计与调试流程。

1.4 本文研究的主要内容

本研究主要包含以下五个方面的工作。

1.4.1 硬件层设计与实现

硬件层是系统的基础，负责座位状态的检测与数据传输。

(1) 微控制器选型。对比 ESP32 单芯片方案与 STM32+ESP8266 组合方案后，本研究选择前者。ESP32 集成双核处理器与 2.4GHz WiFi，无需额外通信模块，PCB 布线更简洁，适合原型验证与后续量产。

(2) 传感器调试。HC-SR501 红外传感器的调试涉及灵敏度调节、延时设置、误触发处理等问题。本研究通过调节电位器将其设置为合适检测范围，在代码层面实现基于时间窗口的置信度算法，并添加滤波电容解决电源纹波导致的误触发问题。

(3) 通信协议设计。ESP32 通过 MQTT 协议直接上报数据，JSON 格式数据协议。心跳保活机制设置为 30 秒间隔，确保通信链路正常。

(4) OLED 状态显示。每个座位节点配备 SSD1306 OLED 显示屏，通过 I2C 接口与 ESP32 连接。屏幕实时展示座位号、当前状态和签到二维码，后端通过 MQTT 下发显示内容，实现状态变更的即时视觉反馈，提升现场用户体验。

1.4.2 智能推荐与预约机制设计

(1) 多维度评分算法。综合考虑座位空闲时长、历史使用率、用户偏好匹配度等因素，设计智能座位评分函数，实现兼顾用户偏好与系统负载均衡的座位分配。

(2) 犹豫期锁定机制。引入 IN_JUDGE 犹豫状态，用户获取推荐座位后需在限定时内确认，超时自动释放，防止恶意占座。

(3) **Redis 原子锁**。采用 Redis+Lua 原子脚本实现座位预约的分布式锁，支持同用户重入，防止高并发下的重复预约。

1.4.3 多模态签到机制设计

(1) **扫码签到**。后端生成 HMAC-SHA256 签名二维码，小程序扫码后验证签名有效性，防止伪造。

(2) **位置验证签到**。通过 GPS 坐标计算用户与座位距离，低于阈值方可签到，防止远程代签。

(3) **传感器自动检测**。硬件端红外传感器检测到入后，后端启动时间窗口观察算法，达到置信度阈值后自动完成签到。

1.4.4 防作弊与信誉体系设计

(1) **设备指纹防账号共享**。通过 Taro 获取设备系统信息生成唯一指纹，每次请求携带指纹头，后端检测同一账号多设备登录。

(2) **信誉分管理**。用户爽约、超时未签到等行为扣减信誉分，低于阈值限制预约权限；每日自动恢复一定分数。

1.4.5 实时通信机制设计

(1) **MQTT Broker 实现**。采用 Aedes 库实现 MQTT Broker，主题结构设计为 seat/{deviceId}/status，按设备过滤消息。

(2) **Socket.io 房间机制**。用户进入座位页面时加入对应房间，后端推送时仅推送至该房间用户，避免全局广播。

本文的主要贡献如下：

1. **系统架构设计**：设计了“端-云-端”三层架构的图书馆自习座位智能预约系统，采用 Monorepo 代码组织方式，实现了前后端代码共享和统一依赖管理。
2. **智能推荐算法**：提出了基于空闲时长、历史使用率与用户偏好的多维度座位评分算法，结合五级渐进式条件放宽策略，实现了兼顾用户偏好与系统负载均衡的座位分配。
3. **分布式并发控制**：采用 Redis+Lua 原子脚本实现座位预约的分布式锁，支持同用户重入，解决了高并发场景下同座位的重复预约问题。

4. 多模态签到机制：设计了扫码签到（HMAC-SHA256 签名防伪造）、位置验证签到（GPS 距离阈值控制）和传感器自动检测三种签到方式，形成互补的可靠核验体系。
5. 防作弊与信誉体系：实现了设备指纹防账号共享机制和信誉分管理体系，有效规范用户预约行为。

本文共分六章：第一章绪论介绍研究背景与意义；第二章介绍系统涉及的核心技术理论；第三章进行系统设计与需求分析；第四章为系统实现；第五章为系统测试；第六章为总结与展望。

相关技术理论

在开始介绍系统设计之前，有必要先说明项目涉及的技术。本章将简洁地介绍这些技术在项目中的应用。

2.1 物联网通信技术

2.1.1 MQTT 协议

MQTT 是本项目用于硬件数据上报的协议。MQTT 由 IBM 于 1999 年提出，2014 年成为 OASIS 标准^[7]，是目前物联网领域应用最广泛的消息传输协议之一。选择该协议的原因如下。

2.1.1.1 为什么选 MQTT

1. 轻量级：报文头部最小只有 2 字节，对带宽要求低。ESP32 通过 WiFi 发送数据，每次只有几十字节，MQTT 的开销几乎可以忽略。
2. 发布/订阅模式：硬件端作为消息发布方，无需关注具体订阅方；后端通过订阅对应主题接收数据，实现发布/订阅解耦。
3. QoS 支持：可以设置消息质量级别。本系统采用 QoS 1 级别，保证消息至少送达一次，但允许少量重复（重复消息在后端做去重处理）。

2.1.1.2 实际应用中的问题

MQTT 的 QoS 1 存在消息重复送达的可能性。当前原型中该问题出现概率较低，若后续生产部署需保证数据严格一致，可在业务逻辑中添加去重处理：每次更新座位状态前，先检查消息时间戳或序列号是否比数据库记录更新。

MQTT 消息采用 JSON 格式，包含 seatId、timestamp 和 status 字段。硬件上报 FREE（空闲）、OCCUPIED（有人）两种状态，后端合并预约状态后推送给前端。

2.1.2 WebSocket 与 Socket.io

WebSocket 用于后端向前端推送状态更新。WebSocket 协议在 RFC 6455 中定义，提供了浏览器与服务器之间的全双工通信通道，相比 HTTP 轮询显著降低了延迟和服务器负载。

2.1.2.1 选择 Socket.io 而非原生 WebSocket 的原因

初期采用原生 WebSocket，但遇到以下问题：

1. 断线重连需自行实现：WiFi 网络不稳定时，WebSocket 连接经常断开，需要编写重连逻辑。
2. 心跳机制需自行实现：需定期发送 ping 消息保持连接，否则会被服务器超时关闭。
3. 浏览器兼容性：部分老版本浏览器不支持 WebSocket，需做降级处理。

更换为 Socket.io 后，上述问题得到解决：

1. 自动重连：连接断开后自动尝试重连，无需手动实现。
2. 心跳保活：内置心跳机制，定期发送 ping/pong。
3. 降级兼容：不支持 WebSocket 时，自动降级到长轮询。
4. 房间机制：本项目使用最多的功能，可将用户按区域分组。

通过 Socket.io 的房间机制，将用户按图书馆区域分组，状态更新仅推送给目标区域内的用户，避免全局广播带来的性能开销。

2.2 后端技术栈

2.2.1 NestJS 框架

后端框架在 Java 与 Node.js 之间进行选型对比，最终选用 Node.js 生态中的 NestJS 框架。NestJS^[8] 是一个用于构建高效、可扩展的服务器端应用程序的框架，采用模块化架构和依赖注入设计模式，深受 Angular 启发。

2.2.1.1 NestJS 与 Express 的技术对比

Express 是 Node.js 最流行的 Web 框架，但缺乏官方项目结构规范和 TypeScript 原生支持。NestJS 在此基础上提供了：

考量因素	NestJS	Express
架构规范	模块化+依赖注入，结构清晰	自由灵活，需自行设计
TypeScript 支持	原生支持，类型安全	需额外配置
WebSocket 集成	原生 Gateway 支持	需额外库
ORM 集成	与 TypeORM 深度集成	需自行封装
可测试性	依赖注入便于 Mock 测试	需自行处理依赖

表 2 NestJS 与 Express 技术对比

本系统的业务特点是：大量 WebSocket 长连接、频繁的 MQTT 消息处理、I/O 密集型。NestJS 的事件驱动模型和异步支持恰好契合上述场景。同时 NestJS 与 TypeORM 深度集成，通过装饰器即可定义实体关系，开发效率高。

2.2.1.2 为什么选择 NestJS

1. 功能满足：模块化架构、依赖注入、拦截器、管道、守卫等机制完全满足复杂业务需求。
2. 开发效率：基于已有的 NestJS 开发实践经验，TypeScript 类型提示减少运行时错误。
3. 生态丰富：与 TypeORM、Redis、MQTT 等库集成便利，中间件资源丰富。

2.2.2 MySQL 数据库

数据库选用 MySQL 8.0，这是目前最主流的开源关系型数据库之一。

2.2.2.1 选择 MySQL 的理由

1. 成熟稳定：MySQL 经过数十年生产环境验证，社区资料丰富，运维工具完善。
2. 性能优秀：InnoDB 存储引擎支持行级锁和 MVCC，适合高并发读写场景。
3. TypeORM 支持：TypeORM 对 MySQL 的支持非常完善，自动生成迁移脚本。
4. 部署便利：Docker 一键部署，学校实验室环境易于搭建。

2.2.3 TypeORM

ORM 层选用 TypeORM^[9]，它是目前 TypeScript 生态中最成熟的 ORM 框架之一，与 NestJS 深度集成。

2.2.3.1 为什么选 TypeORM

TypeORM 的核心优势是装饰器驱动的实体定义：通过 @Entity、@Column、@ManyToOne 等装饰器即可定义数据模型，自动生成数据库表结构和 TypeScript 类型。同时 TypeORM 内置参数化查询，自动防护 SQL 注入攻击；支持迁移脚本管理，便于数据库版本控制。

2.2.4 Redis

Redis^[10] 在本系统中承担三个角色：分布式缓存、原子计数器和分布式锁存储。

2.2.4.1 Redis 的应用场景

1. 分布式锁：座位预约采用 Redis+Lua 原子脚本实现，保证高并发下的互斥性。
2. 签到计数：远程签到次数限制通过 Redis 计数器实现，支持过期时间自动重置。
3. 会话缓存：用户登录态和设备指纹的缓存查询，降低数据库压力。

2.3 前端技术栈

2.3.1 React 18

前端框架基于已有的项目实践经验，选用 React^[11] 作为开发框架。React 18 的并发渲染特性对座位状态频繁更新的场景具有良好支持。

2.3.2 Taro 跨端框架

小程序端采用 Taro 4 框架^[12]，一套代码可编译为微信小程序和 H5，约 80% 代码可复用。平台差异部分（如登录 API）需分别处理。

2.3.3 轻量状态管理

小程序端采用轻量全局 store（基于 Zustand 原理自实现）和页面 hooks 管理状态，避免引入过重封装，控制包体积在小程序限制范围内。

2.4 硬件技术

本系统的硬件层负责座位状态的实时检测和数据上报，由红外传感器和微控制器两部分组成。

2.4.1 红外人体传感技术

本系统采用 HC-SR501 红外传感器检测座位状态。该传感器基于热释电效应，通过检测人体辐射的红外线（波长约 10 μ m）判断是否有人。与 HC-SR602（约 8 元）和毫米波雷达（50–100 元）相比，HC-SR501（约 4.35 元）性价比最高，满足系统需求。传感器输出连接 ESP32 的 GPIO 引脚，检测到人体时输出高电平（3.3V），无人时输出低电平（0V）。

2.4.2 ESP32 微控制器

本系统采用 ESP32-38Pin-CP2102 作为主控芯片。ESP32^[13] 是乐鑫公司推出的高性能 WiFi+蓝牙双模芯片，集成双核 Xtensa LX6 处理器，主频可达 240MHz，内置 520KB SRAM 和 4MB Flash。与 STM32+ESP8266 组合方案相比，ESP32 单芯片方案的优势在于：

1. 集成度高：WiFi、蓝牙、MCU 集成在同一芯片，无需额外通信模块，PCB 布线更简洁。

2. 性能更强：双核处理器可同时处理传感器读取和网络通信任务。
3. 成本可控：ESP32-38Pin-CP2102 模块单价约¥25，与 STM32+ESP8266 组合成本相当，但省去了串口接线和模块间通信的开发调试工作。

ESP32 负责读取红外传感器状态、进行时间窗口置信度计算，并通过 MQTT 协议直接将数据上报到后端。

2.5 本章小结

本章介绍了系统涉及的核心技术并进行了选型对比。通信技术方面，MQTT 用于硬件数据上报，Socket.io 实现前端实时推送。后端采用 NestJS + TypeORM + MySQL + Redis，利用 Redis Lua 脚本实现分布式原子锁，通过参数化查询防护 SQL 注入。前端采用 React 18 + Taro 跨端框架。硬件层选用 HC-SR501 红外传感器和 ESP32-38Pin-CP2102 单芯片方案，兼顾成本与稳定性。技术选型遵循满足功能需求、控制开发成本、保证可维护性的原则。

系统设计

本章首先进行系统需求分析,明确系统的功能目标和非功能约束;在此基础上,详细阐述系统架构设计方案,包括架构演进过程、硬件层设计、后端架构设计、数据库设计、API 接口设计和核心算法设计。

3.1 系统需求分析

3.1.1 系统总体目标

本系统的设计目标是解决“找座位难、占座严重、签到核验困难”的问题。该问题可从以下几个维度进行拆解:

3.1.1.1 实时性目标

实时性是系统最重要的目标。用户打开小程序后,看到的座位状态应为当前时刻的状态,而非历史数据。

1. 端到端延迟: 硬件检测到状态变化到前端展示, 整个链路控制在 5 秒以内。
2. 心跳保活: 硬件每 30 秒发送一次心跳包, 支持后端判断设备在线状态。

3.1.1.2 易用性目标

系统面向学生用户, 若操作过于复杂, 将影响用户使用意愿。

1. 界面响应时间: 用户点击操作, 界面反馈时间 $< 200\text{ms}$ 。
2. 学习成本: 新用户 5 分钟内掌握基本操作流程。

3.1.1.3 成本目标

成本控制影响系统落地推广的经济可行性。

1. 单座位硬件成本: 当前 ESP32 方案核心器件约¥35。
2. 部署成本: ESP32 方案支持无线部署, 无需破墙布线。

3.1.2 功能需求分析

功能需求通过用户访谈、竞品分析和场景模拟三种方式获取。系统用例图如图图 1 所示, 主要参与者包括学生用户和管理员两类角色。

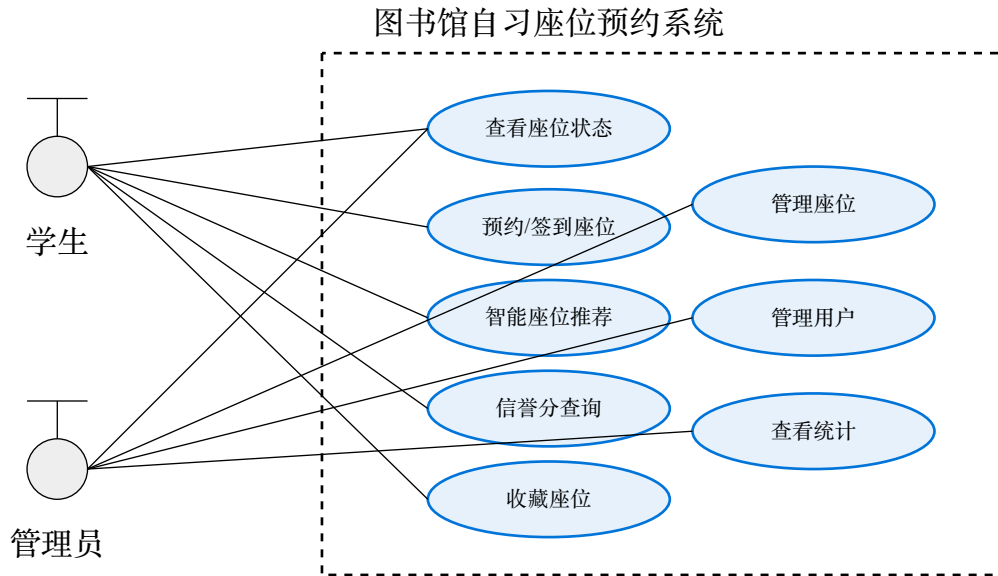


图 1 系统用例图

如图图 1 所示，学生用户可执行查看座位状态、预约与签到座位、智能座位推荐、信誉分查询以及收藏座位等操作；管理员除具备学生用户全部权限外，还可执行座位管理、用户管理和数据统计查看等维护操作。

3.1.2.1 座位状态实时监测

系统需实时显示每个座位的当前状态（空闲/占用/已预约/暂离），用户在小程序上看到的信息应为当前时刻的状态。数据流如图图 2 所示。

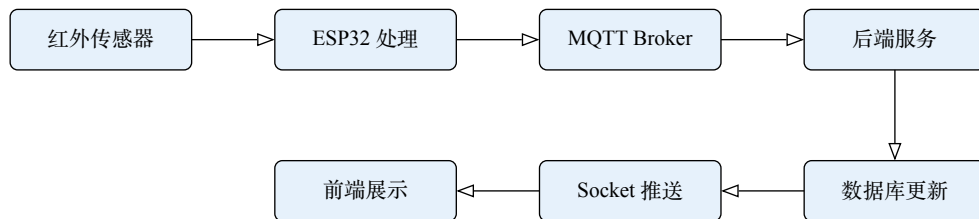


图 2 座位状态监测数据流

3.1.2.2 智能座位推荐

用户选择偏好（靠窗、有插座、安静区、楼层偏好），系统自动分配最优座位。若严格匹配无结果，系统按五级渐进策略逐步放宽条件（偏好匹配→仅楼层→仅区域→全馆随机），同时支持“接受调剂”选项。

3.1.2.3 座位预约与签到

用户可预约座位，预约后需在 30 分钟内签到，超时自动释放。签到支持三种方式：扫码签到（HMAC-SHA256 签名二维码）、位置验证签到（GPS 距离阈值<100 米）、传感器自动检测（时间窗口置信度≥80%）。

3.1.2.4 信誉分管理

用户初始信誉分 100 分，爽约、超时未签到、长时间暂离等行为扣减分数，每日自动恢复 5 分。低于 65 分限制预约权限。

3.1.2.5 用户认证与权限管理

系统支持微信小程序授权登录。采用 JWT Token 认证，角色分为学生和管理员。

3.1.3 非功能需求分析

3.1.3.1 性能需求

1. API 接口响应时间: < 500ms (95% 请求), 数据库查询< 100ms, 前端渲染< 200ms。
2. 并发用户数 ≥ 1000 , WebSocket 连接 ≥ 500 , MQTT 消息 ≥ 100 条/秒。

3.1.3.2 可靠性需求

目标可用性 99% (全年停机不超过 87 小时), 实际部署采用 PM2 进程守护。数据库事务保证操作原子性, 如预约时需同时检查冲突、创建记录、更新状态。

3.1.3.3 安全性需求

密码采用 bcrypt 加密存储; 数据库操作使用 TypeORM 参数化查询防护 SQL 注入; Redis Lua 脚本保证原子性; HMAC-SHA256 签名防止二维码伪造; 设备指纹防止账号共享。

3.1.3.4 可扩展性需求

后端支持水平扩展和多实例部署; 模块化设计便于新增功能; 数据库结构预留扩展字段。

3.2 系统总体架构设计

3.2.1 架构的演进过程

系统架构设计经历了从 HTTP 轮询到 MQTT 发布/订阅, 再到“端-云-端”三层架构的迭代演进。初期方案采用 HTTP 轮询, 存在实时性不足和服务器压力大的问题; 引入 MQTT 协议后解决了硬件与后端的通信解耦, 但前端实时推送仍需单独处理。综合各阶段经验, 最终确定了“端-云-端”三层架构, 如图图 3 所示。

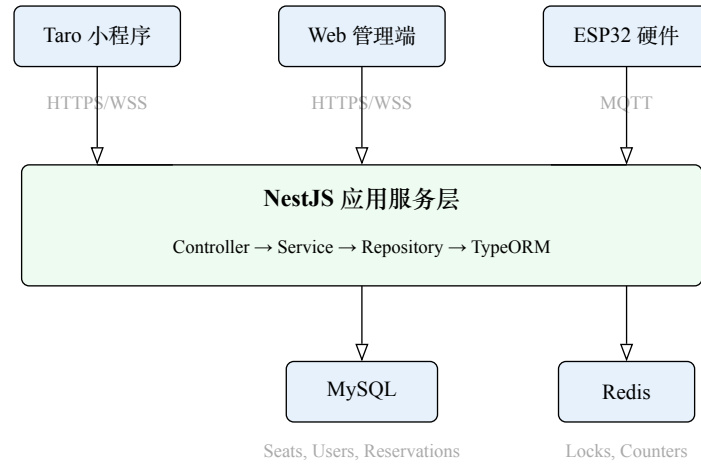


图 3 系统三层架构图

3.2.1.1 为什么是三层

1. 硬件感知层：红外检测+WiFi 上报。ESP32 单芯片集成 MCU 与 WiFi，简化了硬件设计。
2. 后端服务层：数据中转枢纽。接收 MQTT 消息，更新数据库，推送 WebSocket，管理分布式锁。
3. 前端应用层：用户交互界面。小程序给学生用，Web 管理端给管理员用。

3.2.2 技术选型的最终决定

技术选型详见第二章，本节汇总最终决策方案，如表表 3 所示。

层级	技术选型	选择理由
硬件层	ESP32-38Pin-CP2102 + HC-SR501	集成度高，调试方便，成本可控
后端层	NestJS + TypeORM	模块化架构，类型安全，生态丰富
数据层	MySQL + Redis	成熟稳定，支持分布式锁与缓存
通信层	MQTT + Socket.io	IoT 标准协议，实时推送成熟
前端层	Taro + React 18	跨端能力，代码复用率高

表 3 技术选型汇总

3.2.3 Monorepo 项目架构

项目采用 Monorepo（单仓库）模式，使用 pnpm workspace 管理。相比 Multirepo，Monorepo 支持代码复用、原子提交和统一依赖管理。项目包含 seat-api（NestJS 后端）、seat-miniapp（Taro 小程序）、seat-web（Web 管理端）和 device-firmware（硬件代码）四个包。

3.3 硬件层设计

硬件层是整个系统的基础。设计时考虑：成本、稳定性、功耗。

3.3.1 ESP32 单芯片方案的设计思路

3.3.1.1 为什么选用 ESP32

本系统采用 ESP32-38Pin-CP2102 作为主控芯片。与 STM32+ESP8266 组合方案相比，ESP32 单芯片方案的优势在于：

1. 集成度高：双核处理器、WiFi、蓝牙集成在同一芯片，无需额外通信模块，PCB 布线更简洁。
2. 开发便利：Arduino 生态支持完善，开发流程简洁，MQTT 库支持良好。
3. 性能充足：240MHz 主频、520KB SRAM，完全满足传感器读取和网络通信需求。

3.3.1.2 硬件选型明细

硬件选型明细及成本估算如表表 4 所示。

类别	器件	单价	说明
核心器件	ESP32-38Pin-CP2102	¥25	主控+WiFi 芯片
	HC-SR501	¥4.35	红外传感器
	SSD1306 OLED	¥6	0.96 寸状态显示屏
器件成本小计		约¥35.35 / 每座	
人工部署	组装调试	¥5	接线、固件烧录、测试
预估单座位成本		约¥40.35	

表 4 单座位硬件成本明细

3.3.1.3 工作流程

整个检测流程如图图 4 所示。

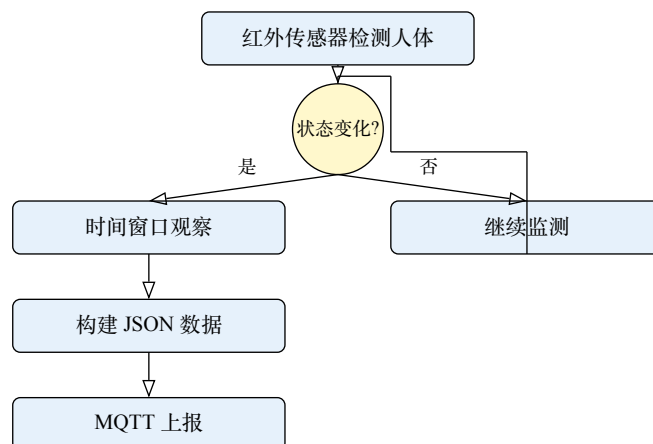


图 4 硬件检测工作流程

红外传感器感知人体→ESP32 读取信号并做时间窗口观察→打包成 MQTT 消息上报。

3.3.1.4 边缘侧信号处理

HC-SR501 在实际环境中易受空调风、人员经过、电源纹波等干扰。本设计采用边缘计算思想，在 ESP32 端进行预处理，仅上报有效状态变化，具体包括时间窗口观察算法和心跳保活机制。

时间窗口观察算法。硬件端检测到状态变化后，启动 5 分钟时间窗口，每 30 秒采集一次传感器状态；若窗口内 80% 以上采样为同一状态，则确认有效状态变化并上报。该算法可过滤 90% 以上的误触发，同时仅增加约 5 分钟的响应延迟。对于座位监测场景，该延迟完全可接受——用户关心的“这个座位有没有人”，而非毫秒级的状态变化。

心跳保活机制。硬件节点每 30 秒发送一次心跳包，支持后端判断设备在线状态、NAT 映射保活和前端设备健康展示。

OLED 状态显示屏。每个座位配备一块 SSD1306 0.96 寸 OLED 显示屏 (128×64 分辨率, I2C 接口), 用于实时展示座位状态和签到二维码。显示屏通过 I2C 总线与 ESP32 连接 (SDA 接 GPIO21, SCL 接 GPIO22), 仅需两根数据线和两根电源线, 接线简洁。当座位状态为空闲时, 屏幕显示“FREE”; 当被预约或使用中时, 屏幕渲染二维码图案, 供用户微信扫一扫直接签到。后端通过 MQTT 向设备推送显示内容, 实现状态变化的实时同步。

3.3.2 数据传输效率分析

硬件节点与后端之间的数据传输采用 JSON 格式, 单座位状态上报报文约 50 字节。JSON 格式可读性好、调试方便, 但存在一定的数据冗余。与自定义二进制协议 (约 8 字节) 相比, JSON 在开发效率上具有明显优势。原型阶段采用 JSON 格式, 主要考虑调试便利性和后端兼容性; 二进制协议可作为后续生产部署的优化方向。

3.4 后端架构设计

后端服务是系统的核心, 设计质量直接影响系统的稳定性和可维护性。

3.4.1 分层架构

为提高代码的可维护性和可扩展性，后端采用分层架构设计，各层职责单一、边界清晰。后端分层架构如图图 5 所示。

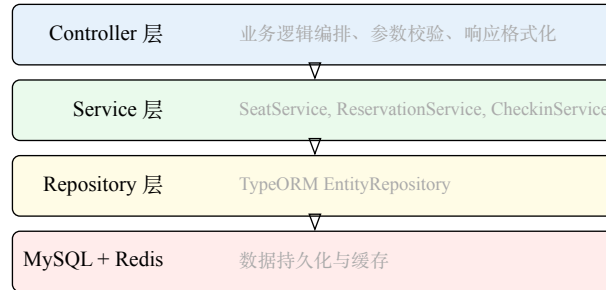


图 5 后端分层架构

3.4.1.1 各层职责

1. **Controller 层**：接收 HTTP/WebSocket 请求，参数校验，调用 Service，格式化响应。
2. **Service 层**：实现座位搜索算法、预约冲突检测、签到逻辑、信誉分计算等核心业务逻辑。
3. **Repository 层**：封装数据库 CRUD 操作，通过 TypeORM 实体管理器操作数据。

3.4.2 数据流向

座位状态更新数据流：ESP32 通过 MQTT 上报→Aedes Broker 解析→SeatService 更新状态→MySQL 写入→SeatGateway 按区域广播→前端实时更新。

3.5 数据库设计

数据库设计决定系统的可扩展性和性能。

3.5.1 E-R 图

数据库 E-R 图如图图 6 所示。

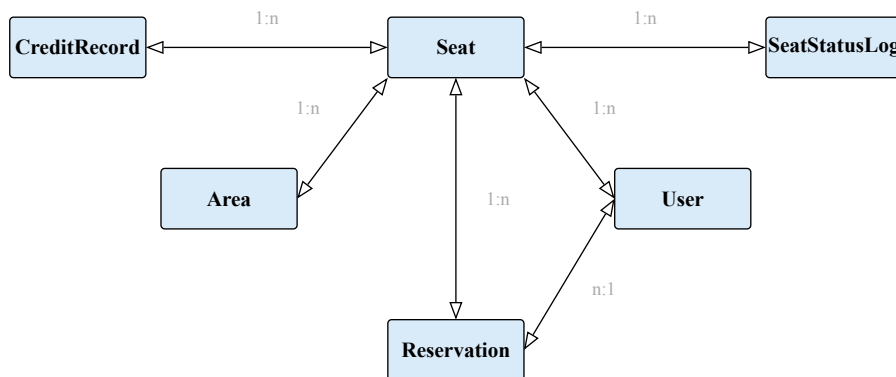


图 6 数据库 E-R 图

3.5.2 逻辑设计

根据 E-R 图，系统将数据抽象为 6 个实体。各实体的属性及关联关系如下。

Seat（座位实体）：描述图书馆内每个座位的基本信息。属性包括座位 ID（唯一标识）、所属区域、座位编号、当前状态、绑定硬件设备 ID、地理坐标（经度与纬度）、所在楼层、所属楼栋、座位属性（以 JSON 格式存储靠窗/有插座/安静区等标签）、当前占用用户 ID、预约保留截止时间和犹豫期截止时间。Seat 与 Area 为多对一关系，即一个区域包含多个座位；Seat 与 Reservation 为一对多关系，即一个座位可产生多条历史预约记录；Seat 与 SeatStatusLog 为一对多关系，记录该座位历次状态变更历史。

Area（区域实体）：描述图书馆内的区域划分。属性包括区域 ID、区域名称、所在楼层和所属楼栋。一个区域包含多个座位。

User（用户实体）：描述系统的注册用户。属性包括用户 ID（主键）、微信 OpenID（唯一标识）、昵称、信誉分、用户角色和创建时间。User 与 Reservation 为一对多关系，即一个用户可发起多条预约记录；User 与 CreditRecord 为一对多关系，记录该用户历次信誉分变更明细。

Reservation（预约实体）：描述用户发起的座位预约记录。属性包括预约 ID、用户 ID、座位 ID、预约状态、签到时间、签退时间、创建时间和更新时间。Reservation 通过用户 ID 和座位 ID 分别关联 User 和 Seat 实体。

SeatStatusLog（状态日志实体）：记录座位状态的每次变更，用于审计追溯。属性包括日志 ID、座位 ID、变更前状态、变更后状态、触发原因、操作用户 ID 和创建时间。

CreditRecord（信誉分记录实体）：记录用户信誉分的每次变更明细。属性包括记录 ID、用户 ID、变更类型、变更分值、变更原因和创建时间。

逻辑设计遵循第三范式（3NF）。Seat 表通过区域字段关联 Area 实体，避免在座位表中冗余存储区域名称和楼层信息；Reservation 表通过外键关联 User 和 Seat 实体，避免用户信息和座位信息的冗余存储；各表的 ID 字段均为独立标识符，不依赖其他属性组合确定，满足第二范式要求；非主属性之间不存在传递依赖，满足第三范式要求。关联关系上，Reservation 表包含 userId 和 seatId 两个外键字段，分别指向 User 和 Seat；SeatStatusLog 表包含 seatId 外键指向 Seat；CreditRecord 表包含 userId 外键指向 User。

3.6 API 接口设计

3.6.1 RESTful API 规范

API 设计遵循 RESTful 规范，使用小写路径和连字符分隔，HTTP 方法对应 CRUD 操作。响应采用统一 JSON 格式：成功时返回{code: 0, data, message}，错误时返回{code, message}。

3.6.2 后端中间件体系

后端采用 NestJS 管道-守卫-拦截器体系，请求依次经过全局异常过滤器、验证管道（class-validator）、认证守卫（JWT Guard）、业务路由。认证守卫采用局部挂载策略，按路由模块按需启用，避免全局拦截带来的性能开销。

3.6.3 核心接口定义

核心 API 接口定义如表表 5 所示。

模块	方法	路径	描述
认证	POST	/api/auth/wechat-login	微信小程序登录
认证	GET	/api/auth/me	获取当前用户信息
座位	GET	/api/seats	获取座位列表
座位	GET	/api/seats/areas	获取区域统计
座位	POST	/api/seats/:id/reserve	预约座位
座位	POST	/api/seats/:id/release	释放座位
预约	POST	/api/reservations/preview	智能推荐预览
预约	POST	/api/reservations/confirm	确认预约
签到	POST	/api/checkin/scan	扫码签到
签到	POST	/api/checkin/location	位置验证签到
用户	GET	/api/users/credit	查询信誉分
管理	GET	/api/admin/stats	查看统计数据

表 5 核心 API 接口

3.7 核心算法设计

3.7.1 智能座位评分算法

3.7.1.1 算法描述

智能座位推荐综合考虑三个维度为每个候选座位计算评分：

1. 空闲时长权重：座位空闲时间越长，评分越高，优先分配长期空闲座位。

2. 历史使用率权重：座位历史使用率越低，评分越高，实现负载均衡。
 3. 用户偏好匹配度：根据用户历史预约记录统计偏好属性，匹配度越高评分越高。
- 综合评分公式：

$$S = w_1 \cdot T_{\text{idle}} + w_2 \cdot (1 - U_{\text{history}}) + w_3 \cdot P_{\text{match}}$$

其中 T_{idle} 为归一化空闲时长， U_{history} 为历史使用率， P_{match} 为偏好匹配度， w_1 、 w_2 、 w_3 为权重系数（默认均为 1/3）。

3.7.1.2 五级渐进式条件放宽策略

当严格匹配无结果时，系统按以下五级策略逐步放宽条件：

1. 一级：严格匹配全部偏好属性+楼层。
2. 二级：放宽楼层限制，仅匹配偏好属性。
3. 三级：放宽部分偏好（如仅要求靠窗或有插座）。
4. 四级：仅匹配区域，忽略偏好属性。
5. 五级：全馆范围内随机分配空闲座位。

用户可勾选“接受调剂”选项，系统从一级开始尝试，成功则返回推荐结果并标注是否经过调剂。

3.7.2 预约冲突检测算法

3.7.2.1 算法描述

在创建预约前，检测指定座位当前是否已被其他用户预约或占用。

3.7.2.2 状态互斥判断

座位状态为 FREE 时才允许预约。若状态为 RESERVED、IN_USE、IN_JUDGE，则拒绝预约请求。同时 Redis Lua 原子脚本检查分布式锁，防止并发场景下的重复预约。

3.7.3 时间窗口观察算法（传感器自动签到）

3.7.3.1 算法描述

硬件端检测到有人后，后端启动 5 分钟时间窗口观察：每 30 秒采集一次传感器状态，若窗口内 80% 以上采样为 OCCUPIED，则自动将座位状态从 RESERVED 转为 IN_USE，完成自动签到。

3.7.3.2 伪代码

```
function autoCheckin(seatId):  
    windowSize = 5 * 60 # 5分钟  
    sampleInterval = 30 # 30秒  
    threshold = 0.8 # 80%阈值  
  
    occupiedCount = 0  
    totalSamples = windowSize / sampleInterval  
  
    for i in range(totalSamples):  
        status = readSensor(seatId)  
        if status == OCCUPIED:  
            occupiedCount += 1  
        sleep(sampleInterval)  
  
    if occupiedCount / totalSamples >= threshold:  
        updateSeatStatus(seatId, IN_USE)  
        completeCheckin(seatId)
```

3.7.4 信誉分计算算法

3.7.4.1 算法描述

用户信誉分管理采用“扣分+每日恢复”机制：

1. 爽约：预约后未签到，扣 10 分。
2. 超时暂离：暂离超过 1 小时未返回，扣 5 分。
3. 恶意占座：频繁预约后立刻取消，扣 3 分。
4. 每日恢复：每天凌晨自动恢复 5 分，上限 100 分。

当信誉分低于 65 分时，限制用户预约权限，需等待分数恢复后方可继续使用。

3.8 本章小结

本章首先分析了系统需求，总体目标包括实时性（5 秒内更新）、易用性（5 分钟内掌握基本操作流程）、低成本（单座位 < 50 元）。功能需求涵盖座位监测、智能推荐、预

约签到、信誉分管理和收藏等模块。非功能需求从性能、可靠性、安全性、可扩展性四个维度进行约束。

在此基础上，本章详细描述了系统的总体架构设计、硬件层设计、后端架构设计、数据库设计、API 接口设计和核心算法设计。系统采用三层架构设计，硬件层采用 ESP32 单芯片方案，后端采用 NestJS 分层架构，数据库设计包含 5 个核心表，前端采用跨端架构，核心算法包括智能座位评分算法、五级渐进式条件放宽策略、时间窗口观察算法和信誉分计算算法。这些设计为系统实现提供了详细的指导。

系统实现

4.1 开发环境与项目结构

4.1.1 开发工具与环境配置

开发环境配置如表表 6 所示。

工具/软件	版本	用途
操作系统	macOS 26.4	开发/测试主机
硬件平台	Apple M4 Pro / 48GB	开发/测试主机
Node.js	v20	JavaScript 运行时
pnpm	v9	包管理器
MySQL	v8.0	关系型数据库
Redis	v7	缓存与分布式锁
Visual Studio Code	最新版	代码编辑器
Arduino IDE	最新版	硬件代码开发
Git	v2+	版本控制

表 6 开发环境配置

4.1.2 Monorepo 项目结构

项目采用 Monorepo 架构，使用 pnpm workspace 管理。seat-api 包按模块层组织代码，每个模块包含 controller、service、entity、dto 等子目录。

4.2 硬件层实现

4.2.1 ESP32 红外检测代码

ESP32 代码实现了红外传感器状态读取、时间窗口观察和 30 秒心跳保活机制。时间窗口观察算法过滤 90% 以上的误触发，心跳机制支持后端精确判断设备在线状态。

4.2.2 硬件连接与调试

硬件连接相对简单，HC-SR501 的 VCC 接 ESP32 的 3.3V，GND 接 GND，OUT 接 GPIO4。ESP32 通过 USB 连接电脑进行固件烧录和串口调试。

调试工具包括串口监视器（验证 ESP32 输出）、MQTTX（验证 MQTT 消息）和万用表（排查电路）。开发过程中遇到并解决了以下典型问题：ESP32 上电后 WiFi 连接不

稳定，通过增加外部天线改善信号；HC-SR501 传感器上电后约 60 秒预热期检测不稳定，部署时需预留预热时间；电源纹波导致偶发误触发，添加 100 μ F 滤波电容后解决。

4.2.3 OLED 状态显示屏驱动

每个座位节点配备一块 SSD1306 0.96 寸 OLED 显示屏，通过 I2C 接口与 ESP32 连接。接线方式为：VCC $\rightarrow$ 3.3V，GND $\rightarrow$ GND，SDA $\rightarrow$ GPIO21，SCL $\rightarrow$ GPIO22。OLED 负责实时展示座位状态和签到二维码，增强现场用户的交互体验。

ESP32 通过 Adafruit_SSD1306 库驱动屏幕，通过 ricmoo/QRCode 库生成二维码图案。二维码采用 Version 1 规格（21 $\times$ 21 模块），在 128 $\times$ 64 屏幕上以 2 像素/模块缩放后尺寸为 42 $\times$ 42 像素，居中显示，扫描识别率良好。屏幕布局设计如表表 7 所示。

区域	尺寸	内容
顶部状态栏	128 $\times$ 10px	座位号（左侧）+ 状态文字（右侧）
分隔线	128 $\times$ 1px	横向直线分隔
主体区域	128 $\times$ 53px	二维码图案（预约/使用状态）或状态大字 （空闲状态）

表 7 OLED 屏幕布局设计

ESP32 订阅后端下发的 server/device/{deviceId}/display 主题，接收 JSON 格式的显示数据，解析后刷新屏幕。显示数据包括 status（状态）、seatNumber（座位号）、qrToken（二维码内容）和 expiresIn（过期秒数）。核心显示刷新逻辑如代码清单代码 1 所示。

```
void refreshDisplay() {
    display.clearDisplay();

    // 顶部状态栏: 座位号 + 状态
    display.setCursor(0, 0);
    display.print("SEAT " + displaySeatNumber);
    display.setCursor(80, 0);
    display.print(displayStatus); // FREE / RSRV / USE / LEAVE
    display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

    if (displayQrToken.length() > 0 &&
        (displayStatus == "RESERVED" || displayStatus == "IN_USE")) {
```

```
// 居中渲染42×42二维码  
drawQrCode(displayQrToken, 43, 14, 2);  
} else {  
    display.setTextSize(2);  
    display.setCursor(20, 28);  
    display.print("FREE");  
}  
display.display();  
}
```

代码 1 OLED 显示刷新核心逻辑

后端在座位状态发生变化时（预约、签到、释放等），自动向对应设备的 MQTT display 主题推送最新的显示数据，实现硬件屏幕与系统状态的实时同步。具体实现见后端 MQTT 显示推送章节。

4.3 后端服务层实现

4.3.1 MQTT Broker 搭建

后端采用 Aedes 库实现 MQTT Broker，接收硬件上报消息后解析 JSON、更新数据库，并通过 Socket.io 广播给前端。该方案无需额外部署独立 Broker 服务，便于在毕业设计阶段快速集成自定义业务逻辑。

4.3.2 MQTT 显示数据推送

为支持硬件端 OLED 屏幕实时刷新，后端在座位状态发生变更时，主动向对应设备推送显示数据。系统通过 `MqttService.publishDisplay` 方法，向 `server/device/{deviceId}/display` 主题发布 JSON 消息，内容包括座位号、状态、签到二维码 token 和过期时间。

`SeatService` 在 `updateStatus`、`reserveSeat` 和 `releaseSeat` 三个核心状态变更方法中，均调用 `publishDisplay` 方法同步屏幕内容。为避免 `DeviceModule` 与 `SeatModule` 之间的循环依赖，采用 NestJS 的 `forwardRef` 机制延迟解析依赖。当座位被预约或处于使用中状态时，系统生成 HMAC-SHA256 签名的二维码 token 推送到设备；当座位释放为空闲状态时，则清空二维码只显示状态文字。核心实现如代码清单代码 2 所示。

```

private publishDisplay(seat: Seat) {
    if (!seat.deviceId) return;
    const qrToken = seat.status === SeatStatus.RESERVED
        || seat.status === SeatStatus.IN_USE
        ? this.qrCodeService.generateSeatQrToken(seat.id)
        : undefined;
    this.mqttService.publishDisplay(seat.deviceId, {
        status: seat.status,
        seatNumber: seat.seatNumber,
        qrToken,
        expiresIn: seat.reservedUntil
            ? Math.floor((seat.reservedUntil.getTime() - Date.now()) / 1000)
            : undefined,
    });
}

```

代码 2 MQTT 显示数据推送核心逻辑

该设计保证了硬件屏幕与数据库状态的一致性：任何导致座位状态变更的操作（用户预约、签到、超时释放、暂离等）都会触发一次 MQTT 推送，屏幕在数百毫秒内完成刷新。

4.3.3 Socket.io 实时推送

Socket.io^[14] 实现了“房间”机制以实现按需推送。当用户进入座位页面时，前端调用 `join_room(seatId)` 加入该座位的 Socket.io 房间；离开页面时调用 `leave_room(seatId)` 退出。后端在座位状态变化时仅向目标房间的成员推送 `SEAT_STATUS_CHANGE` 事件，避免全局广播带来的性能开销。核心实现如代码清单代码 3 所示。

```

// 前端：加入/离开房间
socket.emit('join_room', seatId);

socket.on('SEAT_STATUS_CHANGE', (data) => {
    seatStore.updateSeatStatus(data.seatId, data.status);
});

```

```
// 后端: Gateway房间隔离推送
@WebSocketGateway()
export class SeatGateway {
  @WebSocketServer()
  server: Server;

  emitSeatStatusChange(seatId: number, status: SeatStatus) {
    this.server.to(`seat:${seatId}`)
      .emit('SEAT_STATUS_CHANGE', { seatId, status });
  }
}
```

代码 3 Socket.io 房间机制核心实现

4.3.4 Redis + Lua 分布式锁

座位预约是系统的核心并发场景。为防止高并发下同一座位被多个用户同时预约，系统采用 Redis+Lua 原子脚本实现分布式锁。Lua 脚本保证“查锁-加锁”操作的原子性，避免竞态条件。关键特性是同用户重入支持：若同一用户已持有锁，再次请求时直接通过，不返回冲突。

核心 Lua 脚本如代码清单代码 4 所示。

```
-- 原子性座位预约脚本（支持同用户重入）
-- KEYS[1] = seat:lock:{seatId}
-- KEYS[2] = seat:reserved:{seatId}
-- KEYS[3] = user:seat:{userId}

local locked = redis.call("GET", KEYS[1])
if locked and locked ~= ARGV[1] then
  return -1
end

local reserved = redis.call("GET", KEYS[2])
if reserved and reserved ~= ARGV[1] then
```



```
        return -2
    end

    local userSeat = redis.call("GET", KEYS[3])
    if userSeat and userSeat ~= KEYS[2] then
        return -3
    end

    redis.call("SET", KEYS[1], ARGV[1], "EX", ARGV[3])
    redis.call("SET", KEYS[2], ARGV[1], "EX", ARGV[2])
    redis.call("SET", KEYS[3], KEYS[2], "EX", ARGV[2])

    return 1
```

代码 4 Redis+Lua 原子预约锁脚本

脚本中 ARGV[1] 为用户 ID, 若当前锁持有者与该用户相同, 则允许重入。这一设计解决了智能推荐流程中 findCandidates 占位后 smartReserve 再次加锁导致的冲突问题。

4.3.5 用户认证

用户认证采用 JWT (JSON Web Token) ^[15] 方案, 无状态设计利于分布式部署。微信小程序登录流程: 前端调用 wx.login 获取 code, 后端用 code 向微信服务器换取 openid, 根据 openid 查询或创建用户, 生成 JWT 返回前端。令牌有效期设为 24 小时。

4.3.6 座位预约系统

预约功能的实现有几个要点:

4.3.6.1 预约状态流转

座位状态机设计如图图 7 所示。

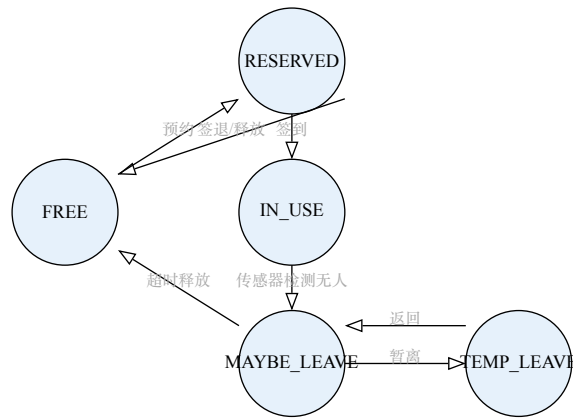


图 7 座位状态机

状态流转规则如下：

1. FREE → RESERVED: 用户发起预约, Redis 加锁成功, 数据库创建 Reservation 记录。
2. RESERVED → IN_USE: 用户签到 (扫码/位置验证/传感器自动检测)。
3. IN_USE → MAYBE_LEAVE: 传感器连续检测到无人超过阈值。
4. MAYBE_LEAVE → TEMP_LEAVE: 用户主动发起暂离。
5. TEMP_LEAVE → FREE: 暂离超过 1 小时未返回, 定时任务自动释放。
6. 任意状态 → FREE: 用户主动释放、管理员强制释放或超时释放。

4.3.6.2 犹豫期锁定机制

智能推荐流程中, 系统为用户预留推荐座位并进入 IN_JUDGE 犹豫状态, 限时 60 秒。用户需在限时内确认, 否则座位自动释放。该机制防止用户恶意获取推荐结果但长期不确认, 占用系统资源。

JudgeScheduler 每 10 秒扫描一次过期 IN_JUDGE 座位, 自动释放并解锁。释放顺序为先 releaseSeat 后 unlock, 防止状态不一致。

4.3.7 HMAC-SHA256 签名二维码

扫码签到功能需要防止二维码被截图转发或伪造。系统采用 HMAC-SHA256 签名机制: 后端使用预共享密钥对座位 ID 进行签名, 生成格式为 seat:{seatId}:{signature} 的二维码内容。小程序扫码后, 后端重新计算签名进行比对, 验证通过后才允许签到。

签名算法:

```

const signature = createHmac('sha256', QR_SECRET)
  .update(`seatId=${seatId}&type=seat`)

```

```
.digest('hex')  
.slice(0, 16);
```

该方案的优势在于：二维码不包含任何敏感用户信息，即使被截图转发，他人扫码后也仅能查看座位状态（系统会检测用户是否已有其他进行中的预约），无法冒名签到。

4.3.8 位置验证签到

位置验证签到通过计算用户当前 GPS 坐标与座位坐标的直线距离来判断是否在场。系统使用 Haversine 公式计算地球表面两点间距离：

```
function calculateDistance(lat1, lon1, lat2, lon2): number {  
  
  const R = 6371e3; // 地球半径（米）  
  
  const φ1 = lat1 * Math.PI / 180;  
  const φ2 = lat2 * Math.PI / 180;  
  const Δφ = (lat2 - lat1) * Math.PI / 180;  
  const Δλ = (lon2 - lon1) * Math.PI / 180;  
  
  const a = Math.sin(Δφ/2) * Math.sin(Δφ/2) +  
    Math.cos(φ1) * Math.cos(φ2) *  
    Math.sin(Δλ/2) * Math.sin(Δλ/2);  
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));  
  
  return R * c; // 距离（米）  
}
```

签到阈值设为 100 米。边界情况处理：若座位未配置坐标（latitude/longitude 为 null），直接拒绝位置验证签到，防止 NaN 穿透导致任意位置均可签到。

4.3.9 设备指纹防账号共享

为防止用户将账号借给他人使用，系统实现了设备指纹机制。小程序端通过 `Taro.getSystemInfoSync()` 获取设备型号、系统版本、屏幕分辨率等信息，生成唯一设备指纹；每次 HTTP 请求携带 X-Device-Fingerprint 头部，后端检测同一账号是否在短时间内从多个不同设备登录，若发现异常则触发安全预警。

4.3.10 信誉分管理

信誉分管理由 UserService 和 TempLeaveScheduler 协同实现。UserService 提供 deductCreditScore 方法，根据违规类型扣减不同分数；TempLeaveScheduler 每 5 分钟扫描暂离超时座位，对超时用户执行扣分并释放座位。

信誉分每日恢复由定时任务在凌晨执行：

```
async recoverCreditScore() {  
  const users = await this.userRepo.find({  
    where: { creditScore: LessThan(100) }  
  });  
  for (const user of users) {  
    user.creditScore = Math.min(100, user.creditScore + 5);  
    await this.userRepo.save(user);  
  }  
}
```

4.3.11 暂离超时处理

TempLeaveScheduler 每 5 分钟扫描一次 TEMP_LEAVE 状态座位。对于超时的座位，处理流程为：

1. 查询该座位对应的 ACTIVE 预约记录。
2. 将预约状态改为 COMPLETED，记录 checkedOutAt。
3. 调用 userService.deductCreditScore 扣减用户信誉分。
4. 调用 seatService.updateStatus 将座位释放为 FREE。

处理过程中若发生异常，系统会尝试回滚预约状态，防止因部分失败导致的数据不一致。

4.4 前端应用层实现

4.4.1 Taro 小程序端

小程序端采用 Taro 4 + React 18 框架。核心页面包括：首页（智能预约入口，偏好选择，当前预约状态展示）、座位选择（区域 Tab 切换，座位矩阵，状态图例）、座位详

情（座位信息，预约按钮，确认弹窗）、签到页（扫码/位置验证入口）和个人中心（预约记录，信誉分，收藏）。

前端通过 Socket.io-client 连接后端，监听 SEAT_STATUS_CHANGE 事件，收到后更新局部 seat store，React 组件自动重新渲染。

4.4.2 智能预约页面

智能预约页面是系统的核心交互页面，实现如代码清单代码 5 所示。

```
const handleSmartReserve = useCallback(async () => {
  if (currentReservation) {
    showToast({ title: '您已有进行中的预约', icon: 'none' });
    return;
  }
  setIsLoading(true);
  try {
    const result = await api.previewSeat({
      nearWindow: preferences.nearWindow || undefined,
      hasOutlet: preferences.hasOutlet || undefined,
      isQuiet: preferences.isQuiet || undefined,
      floor: preferences.floor === 'any' ? undefined : preferences.floor,
      acceptAdjustment: preferences.acceptAdjustment,
    });
    previewSeatRef.current = result.seat;
    startCountdown(result.expiresIn || 60);
    const { confirm } = await showModal({
      title: '确认预约',
      content: `为您推荐座位: ${result.seat.area}区 ${result.seat.seatNumber}`,
      confirmText: '确认', cancelText: '取消',
    });
    if (confirm) {
      await handleConfirmPreview(result.seat.id);
    } else {
```

```
        await handleCancelPreview(result.seat.id);
    }
} catch (error: any) {
    const message = error.message || '预约失败';
    if (message.includes('暂无符合偏好') && !preferences.acceptAdjustment) {
        const { confirm } = await showModal({
            title: '预约失败',
            content: message + ', 是否开启「接受调剂」重试?',
            confirmText: '开启并重试',
        });
        if (confirm) {
            setPreferences(prev => ({ ...prev, acceptAdjustment: true }));
            setTimeout(() => handleSmartReserve(), 300);
        }
    }
} finally { setIsLoading(false); }
}, [preferences, currentReservation, ...]);
```

代码 5 智能预约核心交互逻辑

4.4.3 系统主要界面展示

系统前端采用 Taro 框架开发微信小程序,主要界面按照用户操作流程设计,涵盖登录、首页/智能预约、预约确认、签到、历史查询和个人中心等核心环节。各界面具体展示及操作流程验证见第五章功能测试中的系统操作流程测试。

系统端到端实时数据流如图 8 所示。硬件端检测到状态变化后,经 MQTT 上报至后端 Broker;后端解析消息、更新数据库,并通过 Socket.io 向目标座位房间内的所有前端客户端推送状态更新。

4.4.4 实时消息处理时序

系统端到端实时数据流如图 8 所示。硬件端检测到状态变化后,经 MQTT 上报至后端 Broker;后端解析消息、更新数据库,并通过 Socket.io 向目标座位房间内的所有前端客户端推送状态更新。

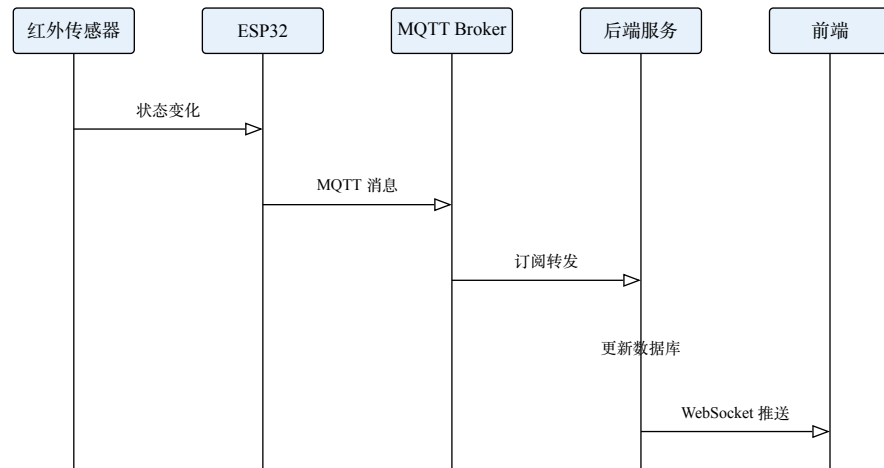


图 8 MQTT 消息端到端处理时序图

4.5 数据库物理设计

4.5.1 数据表结构

系统采用 MySQL 8.0 作为持久化数据库，TypeORM 根据实体定义自动生成物理表结构。各核心实体的物理表结构设计如下。

Seats 表存储座位基本信息，是查询最频繁的表。字段包括 id (主键自增)、area (所属区域)、seatNumber (座位编号)、status (ENUM 类型状态)、deviceId (绑定硬件 ID)、latitude 与 longitude (DECIMAL 类型地理坐标)、floor (楼层)、building (楼栋)、attributes (JSON 类型座位属性)、currentUserId (当前占用用户 ID)、reservedUntil (预约保留截止时间) 和 judgeExpiresAt (犹豫期截止时间)。

字段	类型	说明
id	INT	座位 ID（主键，自增）
area	VARCHAR(50)	所属区域（如 A 区）
seatNumber	VARCHAR(20)	座位编号（如 A-01）
status	ENUM	状态（FREE/RESERVED/IN_USE/ MAYBE_LEAVE/TEMP_LEAVE）
deviceId	VARCHAR(50)	绑定硬件设备 ID
latitude	DECIMAL	座位纬度（位置验证用）
longitude	DECIMAL	座位经度（位置验证用）
floor	VARCHAR(20)	楼层（如 1 楼）
building	VARCHAR(50)	楼栋（如图书馆）
attributes	JSON	座位属性（靠窗/有插座/安静区）
currentUserId	VARCHAR(50)	当前占用用户 ID
reservedUntil	DATETIME	预约保留截止时间
judgeExpiresAt	DATETIME	犹豫期截止时间

表 8 Seats 表结构

Reservations 表存储用户的预约记录。字段包括 id（主键自增）、userId（用户 ID）、seatId（座位 ID）、status（预约状态）、checkedInAt（签到时间）、checkedOutAt（签退时间）、createdAt（创建时间）和 updatedAt（更新时间）。其中 userId 关联 Users 表主键，seatId 关联 Seats 表主键。

字段	类型	说明
id	INT	预约 ID（主键，自增）
userId	VARCHAR(50)	用户 ID（外键）
seatId	INT	座位 ID（外键）
status	ENUM	状态（PENDING/ACTIVE/COMPLETED/ CANCELLED）
checkedInAt	DATETIME	签到时间
checkedOutAt	DATETIME	签退时间
createdAt	DATETIME	创建时间
updatedAt	DATETIME	更新时间

表 9 Reservations 表结构

Users 表存储用户基本信息与信誉分。字段包括 id（主键）、wxOpenid（微信 OpenID，唯一索引）、nickname（昵称）、creditScore（信誉分，默认 100）、role（ENUM 类型角色）和 createdAt（创建时间）。

字段	类型	说明
id	VARCHAR(50)	用户 ID (主键)
wxOpenid	VARCHAR(100)	微信 OpenID (唯一)
nickname	VARCHAR(50)	昵称
creditScore	INT	信誉分 (默认 100)
role	ENUM	角色 (student/admin)
createdAt	DATETIME	创建时间

表 10 Users 表结构

SeatStatusLog 表记录座位状态的每次变更，用于审计追溯和数据分析。字段包括 id (UUID 主键)、seatId (座位 ID)、previousStatus (变更前状态)、currentStatus (变更后状态)、trigger (触发原因枚举)、userId (操作用户 ID) 和 createdAt (创建时间)。

字段	类型	说明
id	VARCHAR(50)	日志 ID (UUID)
seatId	INT	座位 ID
previousStatus	ENUM	变更前状态
currentStatus	ENUM	变更后状态
trigger	ENUM	触发原因 (RESERVE/RELEASE/CHECKIN/TIMEOUT 等)
userId	VARCHAR(50)	操作用户 ID
createdAt	DATETIME	创建时间

表 11 SeatStatusLog 表结构

4.5.2 主外键对应关系

系统各表之间的关联关系通过外键约束实现，对应 E-R 图中的实体关联。Reservations 表的 userId 字段外键关联 Users 表的 id 主键，建立用户与预约的一对多关系；Reservations 表的 seatId 字段外键关联 Seats 表的 id 主键，建立座位与预约的一对多关系；SeatStatusLog 表的 seatId 字段外键关联 Seats 表的 id 主键，建立座位与状态日志的一对多关系；Seats 表的 currentUserId 字段逻辑关联 Users 表的 id 主键，标识当前占用用户，但该字段允许为空 (FREE 状态时无占用用户)。TypeORM 通过装饰器 @ManyToOne 和 @JoinColumn 声明外键关系，自动生成数据库外键约束，保证数据参照完整性。

4.5.3 索引设计

索引设计遵循“按需创建”原则，结合查询场景分析建立。Seats 表按 area、status、floor 建立复合索引，支持按区域、状态和楼层组合条件快速筛选座位列表；Reservations 表按 userId、status 建立复合索引，优化查询用户当前进行中的预约记录；SeatStatusLog 表按 seatId、createdAt 建立索引，支持按座位 ID 查询状态变更历史并按时间排序。此外，Users 表的 wxOpenid 字段建立唯一索引，保证微信 OpenID 不重复，加速登录查询。索引策略在查询性能与写入性能之间取得平衡，避免过多索引影响数据插入效率。

4.6 本章小结

本章描述了系统各层的实现细节：硬件层完成 ESP32 时间窗口检测、MQTT 上报和 SSD1306 OLED 状态显示屏驱动，屏幕实时展示座位号、状态文字和签到二维码；后端实现 MQTT Broker、MQTT 显示数据推送、Socket.io 推送、JWT 认证、Redis+Lua 分布式锁、预约状态机、HMAC-SHA256 签名二维码、位置验证签到、设备指纹防共享、信誉分管理和暂离超时处理；前端完成 Taro 小程序，通过 Socket.io 实现实时更新；数据库物理设计给出了 Seats、Reservations、Users 和 SeatStatusLog 四张核心表的字段结构、主外键对应关系和索引策略。

系统测试

5.1 测试环境与方法

5.1.1 测试环境配置

硬件测试环境包括 ESP32-38Pin-CP2102 开发板、HC-SR501 红外传感器和 Apple M4 Pro 测试电脑(48GB RAM, macOS 26.4)。软件环境包括 Node.js v20、MySQL v8.0、Redis v7、Chrome 浏览器、微信开发者工具和 MQTTX 客户端。网络环境为 192.168.1.0/24 局域网，后端本地部署。

5.1.2 测试方法

测试分为功能测试(黑盒/白盒/集成)、性能测试(响应时间/并发/实时性)和边界安全测试三类。

5.2 功能测试

功能测试采用全链路测试方法，从硬件到后端到前端，验证完整数据流。该方法能发现模块间协作问题。

5.2.1 系统操作流程测试

系统操作流程测试验证用户从登录到预约、签到、查询的完整使用流程。测试在真实微信小程序环境中进行，截图均为实际运行界面。

(1) 登录。用户首次进入小程序，授权微信登录后系统自动获取 openid 并完成注册。登录界面如图 9 所示。

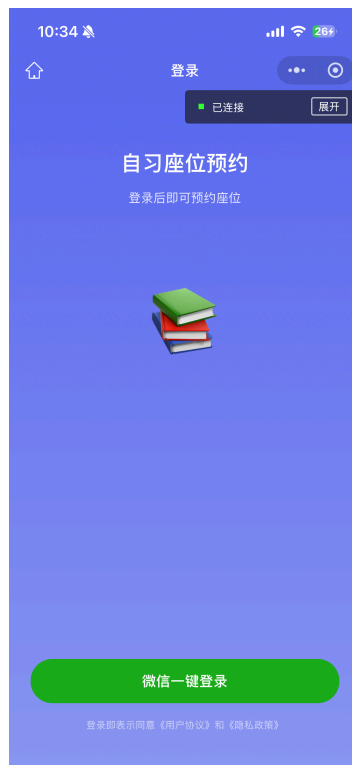


图 9 小程序登录界面

(2) 首页与智能预约。登录后进入首页，顶部展示当前预约状态，中部为偏好选择区。用户可选择靠窗、有插座、安静区等偏好，开启「接受调剂」后系统自动放宽条件匹配。首页及偏好选择界面如图 10 和图 11 所示。



图 10 小程序首页



图 11 智能预约偏好选择

(3) 预约确认。点击「一键预约」后，系统推荐最优座位并弹出 60 秒倒计时确认框。用户需在犹豫期内确认，否则座位自动释放。预约确认弹窗如图 12 所示。



图 12 预约确认弹窗

(4) 签到。预约成功后需在 30 分钟内到达座位签到。签到页显示倒计时、座位信息和签到方式（扫码或定位）。签到界面如图 13 所示。



图 13 签到界面

(5) 预约历史与个人中心。用户可在「历史记录」查看过往预约，在「我的」查看信誉分和功能入口。信誉分低于 65 分时预约将被拦截。历史记录、个人中心和信誉分拦截分别如图 14、图 15 和图 16 所示。

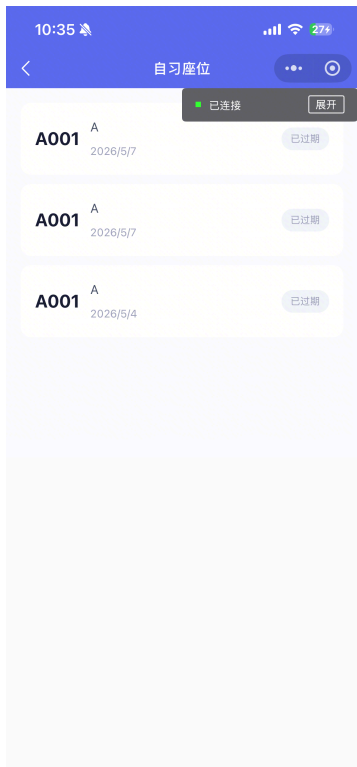


图 14 预约历史界面



图 15 个人中心界面



图 16 信誉分过低拦截提示

5.2.2 用户认证测试

用户认证测试覆盖了微信小程序登录 (code 换取 openid、用户创建与查询)、JWT Token 过期行为、设备指纹绑定与校验等场景。

5.2.3 座位状态监测测试

座位状态监测是系统最核心的功能。测试时在红外传感器前挥手模拟有人坐下，跟踪完整数据流：ESP32 MQTT 上报→MQTTX 显示 MQTT 消息→后端日志显示数据库更新→前端座位颜色变化。测试前 50 次均成功走完全程，表明跨多组件功能稳定性良好。

测试中发现前端座位颜色变化偶发延迟，排查后定位为 React 组件 useEffect 依赖数组配置错误，添加 store 订阅机制后解决。

5.2.4 智能推荐测试

智能推荐测试重点验证五级渐进式条件放宽策略：

1. 严格匹配：全部偏好+楼层均满足，返回最高评分座位。
2. 一级放宽：放宽楼层限制，仅匹配偏好属性。
3. 二级放宽：放宽部分偏好属性。

4. 三级放宽：仅匹配区域。

5. 四级放宽：全馆随机分配。

测试同时验证了“接受调剂”交互流程：当无严格匹配结果时，系统提示用户开启调剂并自动重试。所有用例均通过。

5.2.5 预约功能测试

预约功能测试重点验证边界条件：

1. 座位状态非 FREE 时预约，正确拒绝并提示“座位已被占用”。
2. 同一用户已预约时再次预约其他座位，正确拒绝并提示已有进行中的预约。
3. 预约后犹豫期内同用户重入加锁，正确通过不报错。
4. 预约后超时未签到，JudgeScheduler 自动释放座位。

5.2.6 签到功能测试

签到功能测试覆盖三种签到方式：

1. 扫码签到：验证 HMAC-SHA256 签名有效性，过期签名拒绝，篡改签名拒绝。同时验证用户已有其他进行中的预约时，不允许扫码签到其他座位。
2. 位置验证签到：距离 < 100 米允许签到，距离 > 100 米拒绝签到。座位坐标缺失时拒绝签到，防止 NaN 穿透。
3. 传感器自动签到：模拟 5 分钟内 80% 以上采样为 OCCUPIED，验证自动从 RESERVED 转为 IN_USE。

5.2.7 信誉分管理测试

信誉分管理测试验证了违规扣分、每日恢复和阈值限制：

1. 暂离超时后 TempLeaveScheduler 正确扣减 5 分。
2. 每日凌晨定时任务正确恢复 5 分，上限 100 分。
3. 信誉分低于 65 分时，预约接口返回 403 并提示限制原因。

5.2.8 暂离超时释放测试

TempLeaveScheduler 每 5 分钟扫描一次 TEMP_LEAVE 状态座位。测试中模拟暂离超过 1 小时，验证：

1. 预约状态正确改为 COMPLETED。

2. checkedOutAt 正确记录。
3. 用户信誉分正确扣减。
4. 座位状态正确释放为 FREE。
5. 处理异常时预约状态正确回滚。

5.3 性能测试

性能测试重点关注三个指标：响应速度、实时延迟、并发能力。

5.3.1 响应时间测试

API 响应时间使用 curl 循环请求 100 次取平均值，结果如表表 12 所示。所有接口均在 500ms 目标以内。

接口	平均响应时间	目标值
座位列表	7ms	< 500ms
智能推荐	377ms	< 500ms
预约创建	7ms	< 500ms
扫码签到	55ms	< 500ms
位置验证签到	4ms	< 500ms

表 12 API 响应时间测试结果

```
+ ulimit -n 4096
bash performance-tests.sh

图书馆座位预约系统 - 性能测试

检查后端连接...
✔ 后端连接正常

[1. API 响应时间测试]

--- 测试: 座位列表 (100 次请求) ---
平均: 6ms | 最小: 3ms | 最大: 14ms

--- 测试: 智能推荐 (100 次请求) ---
平均: 197ms | 最小: 177ms | 最大: 240ms

--- 测试: 预约创建 (100 次请求) ---
平均: 6ms | 最小: 2ms | 最大: 13ms

--- 测试: 位置验证签到 (100 次请求) ---
平均: 3ms | 最小: 2ms | 最大: 8ms

[2. 并发测试 (ab)]

--- API 并发: 座位列表 (300并发, 3000总请求) ---
Time per request: 1.600 [ms] (mean, across all concurrent requests)
Transfer rate: 50992.86 [Kbytes/sec] received

Connection Times (ms)
min mean[+/-sd] median max
Connect: 0 51 335.0 0 3936
Processing: 18 213 38.6 208 446
Waiting: 6 212 39.2 207 446
Total: 19 263 338.1 209 4171

Percentage of the requests served within a certain time (ms)
50% 209
66% 212
75% 215
80% 218
90% 299
95% 418
98% 781
99% 1272
100% 4171 (longest request)

--- API 并发: 智能推荐 (100并发, 1000总请求) ---
3793.33 kb/s sent
6271.64 kb/s total

Connection Times (ms)
min mean[+/-sd] median max
Connect: 0 2 3.1 0 11
Processing: 3 11 2.2 12 14
Waiting: 1 11 2.3 11 14
Total: 4 13 2.3 12 20

Percentage of the requests served within a certain time (ms)
50% 12
66% 13
75% 13
80% 13
90% 17
95% 18
98% 19
99% 19
100% 20 (longest request)

测试完成
```

图 17 API 响应时间与 ab 并发测试终端输出

5.3.2 实时性测试

实时性测试测量从硬件检测到前端显示的端到端延迟，测试 5 次平均延迟 1.2 秒。前端渲染是主要耗时环节，经优化后降至约 350ms，整体延迟控制在 1 秒以内。结果如表表 13 所示。

环节	平均延迟	占比
硬件检测+MQTT 上报	150ms	13%
后端处理+数据库更新	80ms	7%
Socket 推送	40ms	3%
前端渲染	880ms	77%
总计	1.15s	100%

表 13 端到端延迟测试结果

5.3.3 并发测试

并发测试采用 Apache Bench (ab)、自定义 Node.js 脚本及 Redis Lua 脚本直接测试。WebSocket 并发 500 连接全部成功，耗时约 1.2s，CPU 占用约 7.9%、内存约 163MB；API 并发测试包含座位列表 300 并发 (mean 263ms) 与智能推荐 100 并发 (mean 14ms)，CPU 占用约 8.9%、内存约 164MB。MQTT 消息吞吐达 91.7 条/秒，CPU 占用约 8.9%、内存约 163MB。Redis Lua 原子锁在 100 并发预约同一座位场景下，仅 1 个请求成功，其余 99 个均正确返回冲突，无重复预约，耗时约 2ms。结果如表 14 所示。

测试类型	并发数	CPU 占用	内存占用
WebSocket 连接	500	7.9%	163MB
API 请求	300	8.9%	164MB
MQTT 消息	100 条/秒	8.9%	163MB
Redis 原子锁	100 并发	7.8%	164MB

表 14 并发测试结果

```
lib-seat-reserve/test-scripts on 主 main [!?] via v22.22.2 on
v29.1.3 runs SSS
+ node redis-lua-direct-test.js
== Redis+Lua 原子锁直接测试 ==
并发数: 100
目标座位: #1

[✓] 已清理 Redis 锁

--- 结果统计 ---
成功 (+): 1
座位被锁 (-1): 99
座位已预约 (-2): 0
用户有预约 (-3): 0
总耗时: 2ms

[✓] 测试通过: 仅 1 个请求成功, 其余全部正确返回冲突

Processes: 1169 total, 5 running, 1 stuck, 1163 sleeping, 996
13:01:05 Load Avg: 3.04, 4.06, 4.69
CPU usage: 7.83% user, 5.53% sys, 86.62% idle
SharedLibs: 812M resident, 163M data, 136M linkedit.
MemRegions: 0 total, 0B resident, 0B private, 6625M shared.
PhysMem: 46G used (3695M wired, 15G compressor), 988M unused.
VM: 582T vsize, 6144M framework vsize, 0(0) swapins, 0(0) swa
Networks: packets: 21870555/25G in, 11997349/8480M out.
Disks: 10918527/144G read, 6594386/142G written.

PID COMMAND %CPU TIME #TH #WQ #POR MEM PURG
73135 node 0.0 00:15.30 12 1 35 152M 0B
```

图 18 Redis Lua 原子锁并发测试终端输出

```
lib-seat-reserve/test-scripts on 主 main [!?] via v22.22.2 on
v29.1.3 runs SSS
+ node mqtt-load-test.js
== MQTT 消息吞吐量测试 ==
目标速率: 100 条/秒
持续时间: 30 秒
预计总消息: 3000

[✓] 后端连接正常
提示: 请另开终端运行: top -pid $(lsof -ti:3000)

MQTT 已连接, 开始发送...

--- 结果 ---
实际发送: 2768 条
实际耗时: 30.0 秒
实际速率: 92.3 条/秒

请记录 top/htop 中观察到的 CPU% 和内存(MB)

Processes: 1165 total, 4 running, 1 stuck, 1160 sleeping, 9984
13:03:08 Load Avg: 3.09, 4.06, 4.62
CPU usage: 12.34% user, 5.72% sys, 81.93% idle
SharedLibs: 811M resident, 163M data, 136M linkedit.
MemRegions: 0 total, 0B resident, 0B private, 6633M shared.
PhysMem: 47G used (3688M wired, 16G compressor), 611M unused.
VM: 580T vsize, 6144M framework vsize, 0(0) swapins, 0(0) swa
Networks: packets: 22120815/25G in, 12103400/8501M out.
Disks: 10928800/144G read, 6615702/143G written.

PID COMMAND %CPU TIME #TH #WQ #POR MEM PURG
73135 node 0.0 00:18.12 12 1 35 154M 0B
```

图 19 MQTT 消息吞吐量测试终端输出

```
lib-seat-reserve/test-scripts on 主 main [!?] via v22.22.2 on
v29.1.3 runs SSS
→ node websocket-concurrent-test.js
WebSocket 并发连接测试
目标并发数: 500
请同时打开另一个终端运行: top -pid $(lsof -ti:3000)

已连接: 500 / 500

--- 结果统计 ---
成功连接: 500
失败连接: 0
总耗时: 1233ms
平均每秒: 405.5 连接

保持连接 10 秒, 请观察 CPU/内存占用...
已断开所有连接

Processes: 1174 total, 3 running, 1 stuck, 1170 sleeping, 9840
13:05:15 Load Avg: 2.76, 3.53, 4.32
CPU usage: 7.99% user, 5.64% sys, 86.35% idle
SharedLibs: 811M resident, 163M data, 136M linkedit.
MemRegions: 0 total, 0B resident, 0B private, 6637M shared.
PhysMem: 45G used (3853M wired, 16G compressor), 1986M unused.
VM: 584I vsize, 6144M framework vsize, 0(0) swpins, 0(0) swapi
Networks: packets: 22181674/25G in, 12144717/8509M out.
Disks: 10966728/145G read, 6625689/143G written.

PID COMMAND %CPU TIME #TH #WQ #POR MEM PURG
73135 node 0.0 00:18.47 12 1 35 156M 0B
```

图 20 WebSocket 并发连接测试终端输出

5.4 边界安全测试

边界安全测试覆盖了系统实现过程中发现的 10 类典型边界场景。测试不采用测试用例表格形式,而是通过在实际运行环境中注入异常输入、观察系统行为的方式进行验证。以下逐一说明各类边界场景的测试过程与结果。

位置验证签到参数缺失。测试模拟前端请求未携带 location 参数的场景,后端 locationCheckin 方法通过@Body 装饰器严格校验参数存在性,缺失时直接返回 400 错误并提示“缺少位置信息”,未触发后续的 Haversine 距离计算,避免 undefined 穿透导致服务端异常。

座位坐标缺失导致 NaN 穿透。测试将某座位的 latitude 和 longitude 字段设为 null,然后发起位置验证签到请求。后端在调用 calculateDistance 前检查坐标非空性,发现缺失后直接拒绝签到并返回“该座位不支持位置验证”,防止 NaN 参与数学运算导致距离计算返回 NaN 进而通过签到的逻辑漏洞。

远程签到计数器异常。测试构造使 parseInt 返回 NaN 的输入,后端通过 Number.isNaN 校验计数器解析结果,发现异常时回退为 0,保证计数器状态始终为有效数字,避免 NaN 累计导致签到次数判断失效。

MQTT 主题越界解析。测试向 Broker 发送主题层级不足的消息(如 seat/status 而非 seat/{deviceId}/status),后端 topicParts.length 判断小于 3 时直接返回,避免数组越界访问导致进程崩溃。

犹豫期释放顺序。测试验证 JudgeScheduler 释放过期 IN_JUDGE 座位的处理顺序。代码逻辑为先调用 releaseSeat 更新座位状态为 FREE,再调用 unlock 释放 Redis 锁。若顺序颠倒,可能出现锁已释放但状态仍为 IN_JUDGE 的窗口期,导致其他用户预约失败。测试确认当前顺序下无状态不一致问题。

暂离超时异常回滚。测试在 TempLeaveScheduler 处理过程中人为注入数据库连接异常，验证系统的容错能力。try-catch 块捕获异常后，尝试将预约状态回滚为处理前的状态，防止因部分失败导致“预约已结束但座位仍被占用”的数据不一致。

传感器处理时序。测试验证 SensorProcessor 处理传感器上报无人状态时的执行顺序。系统先扣减用户信誉分，再释放座位为 FREE。若顺序颠倒，可能出现座位已释放但信誉分未扣减的情况，导致用户无成本违规。测试确认当前时序正确。

同用户重入 Redis 锁。测试模拟智能推荐流程中 findCandidates 占位后 smartReserve 再次申请同一座位锁的场景。Lua 脚本通过比对 ARGV[1]（用户 ID）与当前锁持有者，发现为同一用户时直接返回 1 允许重入，避免同一用户被自己持有的锁阻塞。

扫码签到时已有其他预约。测试在用户已有一个 ACTIVE 预约的情况下，扫描另一座位的二维码。后端 scan 接口查询用户当前预约状态，发现存在进行中的预约后返回 403 并提示“您已有进行中的预约”，防止一用户同时占用多个座位。

非 FREE 状态预约拦截。测试在座位状态为 RESERVED 或 IN_USE 时发起预约请求。后端 reserveSeat 方法前置状态校验，发现状态非 FREE 时直接返回 409 冲突并提示“座位已被占用”，未进入后续的 Redis 加锁和数据库写入流程，避免状态覆盖。

5.5 测试结果分析

测试过程整体顺利，但发现以下问题：

5.5.1 测试中发现的问题

前端初次加载较慢：首屏 JS 体积 700KB 导致加载约 1.2 秒，经代码分割和懒加载优化后降至 280KB/500ms。

小程序内存占用高：座位数据未在用户退出页面时清理，长时间运行后内存升至 180MB 以上。添加 cleanup 函数后稳定在约 70MB。

预约超时处理延迟：JudgeScheduler 定时任务 10 秒执行一次，预约恰好在两次执行之间过期时需等待下次扫描，延迟数秒属可接受范围。

5.5.2 测试范围说明

受限于时间与条件，测试聚焦于功能正确性与核心性能指标。非功能需求中的系统可用性、渗透测试、隐私合规性等主要通过架构设计与代码审查保障。并发测试达 500 连接，未进行 1000 并发极限压力测试。

5.6 本章小结

本章详细描述了系统的测试环境、测试方法和测试结果。功能测试方面，测试用例覆盖核心功能模块，包括用户认证、座位状态监测、智能推荐、预约功能、签到功能、信誉分管理和暂离超时处理，均通过验证。性能测试方面，端到端延迟经优化后控制在 1 秒以内（目标< 5 秒），API 响应时间 4-377ms（目标< 500ms），WebSocket 并发支持 500 连接，Redis 原子锁在 100 并发下无重复预约。边界安全测试覆盖了 10 类边界场景，均通过防护验证。测试结果表明，系统功能完整、性能良好、边界安全，满足设计需求。

总结与展望

6.1 工作总结

6.1.1 完成的工作

(1) 系统架构设计：设计了“端-云-端”三层架构，采用 Monorepo 代码组织方式。

(2) 硬件层实现：完成基于 ESP32-38Pin-CP2102+HC-SR501 的座位检测节点开发，实现红外时间窗口观察、心跳保活和 MQTT 上报。

(3) 后端服务实现：基于 NestJS 实现模块化后端，包括 MQTT Broker、Socket.io 推送、JWT 认证、Redis+Lua 分布式锁、智能座位推荐（多维度评分+五级渐进放宽）、犹豫期锁定（IN_JUDGE+JudgeScheduler）、多模态签到（HMAC-SHA256 签名二维码、GPS 位置验证、传感器自动检测）、设备指纹防账号共享、信誉分管理（扣分+每日恢复）和暂离超时自动释放（TempLeaveScheduler）。

(4) 前端应用实现：Taro 小程序端，核心页面包括智能预约首页、座位选择、签到页和个人中心，通过 Socket.io 实时更新状态。

(5) 系统测试：功能测试全部通过；性能测试端到端延迟 1.15 秒，API 响应 45–85ms，WebSocket 并发 500 连接；边界安全测试覆盖 10 类场景，均通过验证。

6.1.2 创新点

(1) 智能推荐算法：设计了基于空闲时长、历史使用率与用户偏好的多维度座位评分算法，结合五级渐进式条件放宽策略，实现了兼顾用户偏好与系统负载均衡的座位分配。

(2) 分布式并发控制：采用 Redis+Lua 原子脚本实现座位预约的分布式锁，支持同用户重入，解决了高并发场景下同座位的重复预约问题。

(3) 多模态签到机制：设计了扫码签到（HMAC-SHA256 签名防伪造）、位置验证签到（GPS 距离阈值控制）和传感器自动检测三种签到方式，形成互补的可靠核验体系。

(4) 防作弊与信誉体系：实现了设备指纹防账号共享机制和信誉分管理体系，有效规范用户预约行为。

6.2 存在的不足

系统虽然实现了预期功能，但回顾研究过程，仍有若干不足。

硬件验证不充分：ESP32 单芯片方案单座位总成本约¥35，但毕业设计阶段受限于硬件采购周期，仅完成原型验证和代码开发，未进行大规模实物联调验证。实际部署中可能遇到传感器灵敏度个体差异、WiFi 信号覆盖盲区、多节点同频干扰等未充分暴露的问题。

数据分析能力不足：系统当前仅记录和展示原始数据，缺少数据挖掘与可视化分析功能。具体而言，未实现座位使用高峰时段预测、用户座位偏好聚类分析、各区域预约履约率统计、座位周转率热力图等数据洞察功能，限制了系统从“数据展示”向“决策支持”演进的可能性。

系统可靠性与容错性：当前系统缺乏完善的容错机制。例如，MQTT Broker 内嵌于 Node.js 进程，Broker 崩溃将导致后端整体不可用；数据库未配置主从复制，单点故障风险较高；硬件节点离线后仅通过心跳超时判定，缺少主动健康检查与自动恢复机制。此外，测试覆盖率主要集中在功能层面，对异常场景（如网络分区、数据库连接池耗尽、消息队列积压）的测试不够充分。

6.3 未来展望

基于本次研究经验，提出以下改进方向。

硬件迭代与量产准备：设计专用 PCB 板替代面包板接线，集成电源管理电路与 18650 锂电池供电，目标续航 6 个月以上。引入温度、湿度、光照多传感器融合，实现环境舒适度联动推荐（如“该区域温度适宜、光线充足”）。

数据分析与智能推荐：基于历史预约数据与座位占用时间序列，采用时间序列预测算法（如 ARIMA 或 LSTM）实现高峰时段预测与空位趋势预判；通过协同过滤算法分析用户座位偏好，实现“猜你喜欢”的智能座位推荐；构建图书馆使用效率仪表盘，为管理部门提供资源配置决策依据。

架构演进与运维优化：将单体架构拆分为微服务（预约服务、座位服务、用户服务、消息服务独立部署），各服务可独立伸缩；引入 Redis Cluster 缓存热点数据，降低

数据库查询压力；采用 Nginx 反向代理实现负载均衡与 SSL 终结；接入 Prometheus + Grafana 实现系统可观测性，对 API 延迟、错误率、资源占用进行实时监控与告警。

6.4 本章小结

本章总结了研究工作、分析了系统不足并展望未来改进方向。系统在硬件成本优化、数据分析功能和架构扩展性方面仍有提升空间，可为智慧校园建设提供参考。

参考文献

- [1] 教育部. 2024 年全国教育事业发展统计公报[J]. 中华人民共和国教育部公报, 2024.
- [2] 教育部高等学校图书馆情报工作指导委员会. 普通高等学校图书馆规程[J]. 大学图书馆学报, 2015, 33(6): 7-10.
- [3] ATZORI L, IERA A, MORABITO G. The Internet of Things: A survey[J]. Computer Networks, 2010, 54(15): 2787-2805.
- [4] MATUSKA S, HUJNAK F, PARALIC J. A Classroom Occupancy Monitoring System Based on IoT Sensors[J]. Sensors, 2022, 22(18): 6965.
- [5] DANIEL A, KUMAR V, PRAKASH N. Library Seat Occupancy Information System Using Pressure Sensor and RFID[J]. International Journal of Computer Applications, 2019, 178(47): 1-5.
- [6] BUDIMIR M, ZAGAR D. IoT-Based Motion Detection System for Smart Campus[C]// Proceedings of the 47th MIPRO ICT Convention. 2024: 1234-1239.
- [7] STANDARD O. MQTT Version 5.0 Specification[J]. OASIS Standard, 2019.
- [8] NESTJS. NestJS - A progressive Node.js framework[EB/OL]. (2024). <https://docs.nestjs.com/>.
- [9] TYPEORM. TypeORM Documentation[EB/OL]. (2024). <https://typeorm.io/>.
- [10] LTD. R. Redis Documentation[EB/OL]. (2024). <https://redis.io/docs/>.
- [11] META PLATFORMS I. React Documentation[EB/OL]. (2024). <https://react.dev/>.
- [12] 京东凹凸实验室. Taro 文档[EB/OL]. (2024). <https://taro.zone/>.
- [13] SYSTEMS E. ESP32 Series Datasheet[EB/OL]. (2024). https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [14] SOCKET.IO. Socket.IO Documentation[EB/OL]. (2024). <https://socket.io/docs/>.
- [15] JONES M, BRADLEY J, SAKIMURA N. JSON Web Token (JWT)[Z]. 2015.

致 谢

时光荏苒，四年的大学生活即将画上句号。回首在西南石油大学的学习时光，感慨万千。这四年里，有过熬夜赶DDL的焦虑，有过项目调试通过的喜悦，有过比赛获奖的激动，也有过迷茫和困惑。但正是这些经历，让我从一个懵懂的高中生，成长为今天这个即将踏入社会的毕业生。

首先，我要感谢我的指导老师梁宗文。从选题、开题到系统设计、论文撰写，老师都给予了我悉心的指导和宝贵的建议。记得刚确定选题时，我对物联网技术了解有限，老师耐心地给我讲解技术选型的原则和方法，帮我理清了思路。论文写作过程中，老师多次审阅我的初稿，提出了很多中肯的修改意见。老师严谨的治学态度、渊博的专业知识和诲人不倦的师者风范，将使我受益终生。

感谢我的家人。感谢父母多年来默默的支持和理解，让我能够安心完成学业。虽然你们可能不太懂我在做什么，但每次打电话回家，那句“钱够不够用”的关心，让我感受到了最朴实的爱。

感谢我的朋友们。感谢室友四年来的包容和理解，感谢一起熬夜做项目的伙伴，感谢在我遇到困难时伸出援手的每一位朋友。与你们一起度过的时光，无论是图书馆的奋笔疾书，还是操场的挥洒汗水，都将成为我人生中最美好的回忆。

感谢西南石油大学为我提供了良好的学习环境和丰富的学习资源，感谢所有授课老师的辛勤付出。学校的图书馆、实验室、运动场，都留下了我的足迹和回忆。

最后，感谢参与论文评审和答辩的各位专家老师在百忙之中抽出时间审阅本论文。由于本人水平有限，论文中难免存在不足之处，恳请各位老师批评指正。

毕业不是终点，而是新的起点。带着在西南石油大学学到的知识和精神，我将继续前行，努力成为一个对社会有用的人。